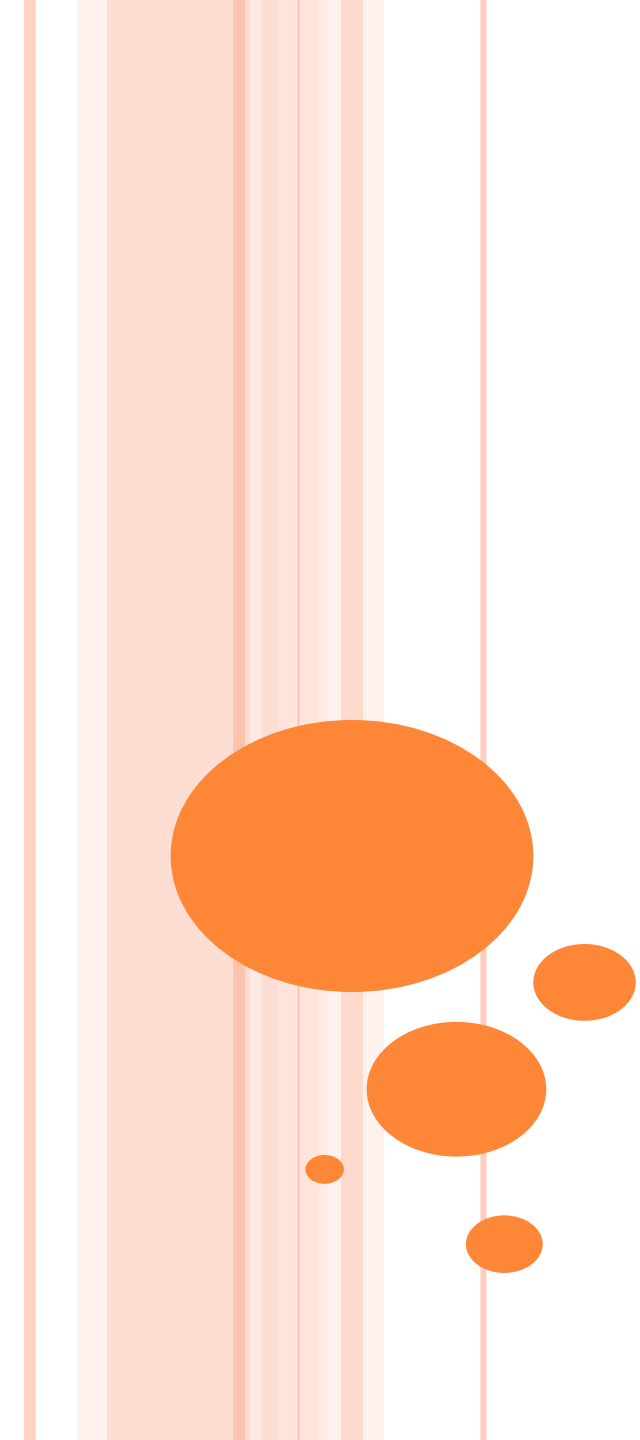




CSE 107: OBJECT ORIENTED PROGRAMMING LANGUAGE

Dr. Tanzima Hashem
Professor
CSE, BUET

TEXTBOOK

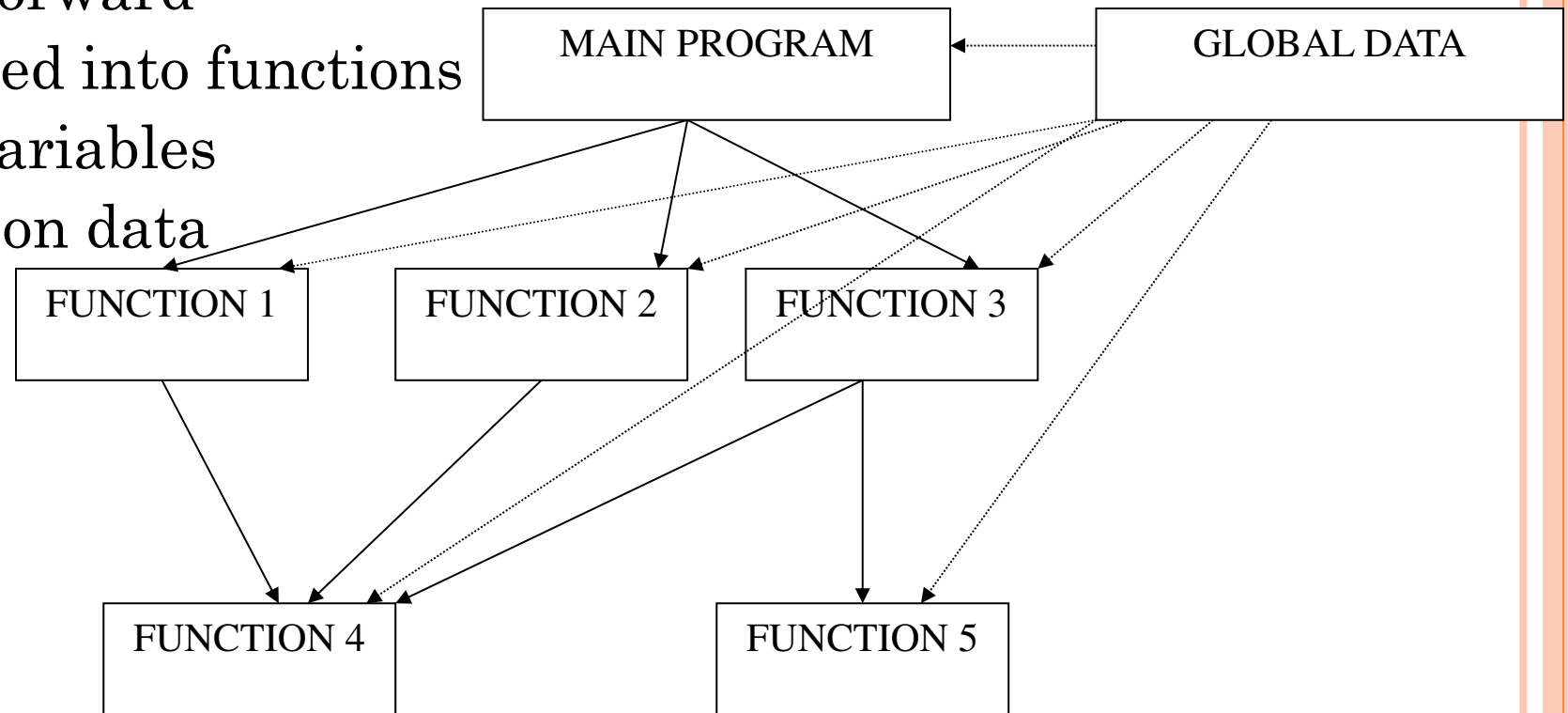
- Teach Yourself C++, Third Edition.
 - Herbert Schildt,
- Tentative Plan
 - Week 1-7 ... C++
 - Week 8-15 ... Java

PROGRAM

- A program is a set of instructions that tells the computer what to do to come up with a solution for a particular problem
- Programming Paradigms
 - Procedural/Structured Programming
 - Focus on functions
 - Object Oriented Programming
 - Focus on objects (data)

PROCEDURAL PROGRAMMING

- Simple and Straightforward
 - A program is divided into functions
 - Data is stored in variables
 - Functions operate on data



- Problems with procedural programming appear as programs grow larger and more complex

OBJECT ORIENTED PROGRAMMING (OOP)

- Object models a real life object
 - Better organization of the code
 - Smaller code
 - Reuse of code
- It organizes a program around its data (that is, objects) and a set of well-defined interfaces to that data

OOP BUILDING BLOCKS: OBJECT AND CLASS

- Object is a unit where variables and functions are combined
- Class is a blueprint from which an object is created
- A class defines the structure and behavior (data and code) that will be shared by a set of objects
- A class is a logical construct; an object has physical reality
- Objects are sometimes referred to as instances of a class

OOP BUILDING BLOCKS: OBJECT AND CLASS

- Class has
 - attributes or properties that describes every object
 - behavior or actions that every object can perform
- Object has
 - data (value for each of its attribute)
 - actions that it can perform
 - an identity

OOP BUILDING BLOCKS: OBJECT AND CLASS

CAR

model
color
cylinder

start()
stop()
move()

FordCar1

model: mustang
color: yellow
cylinder: 6

start()
stop()
move()

FordCar2

model: focus
color: orange
cylinder: 4

start()
stop()
move()

FEATURES

- Abstraction
- Encapsulation
- Inheritance
- Polymorphism

ABSTRACTION

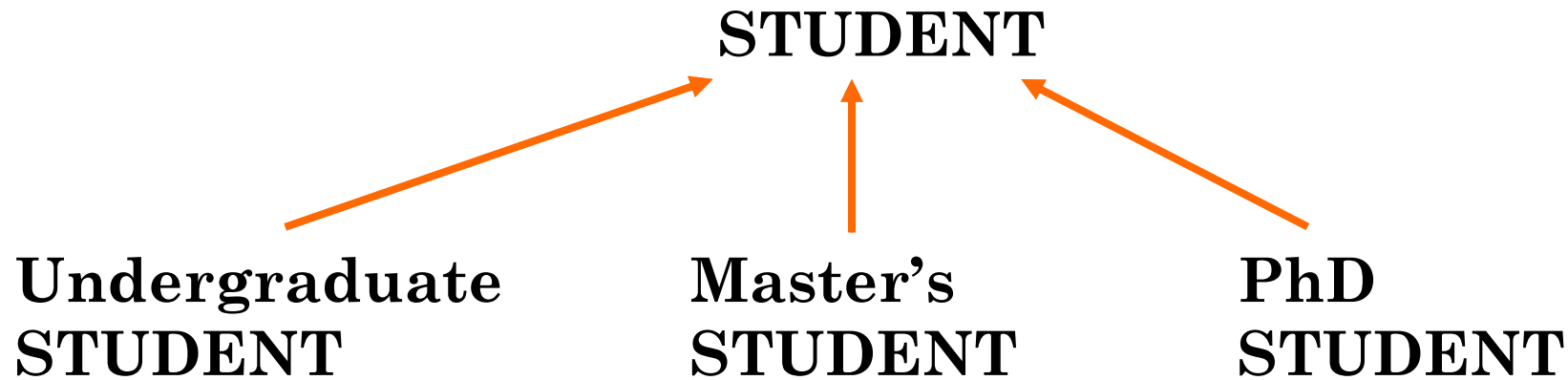
- Humans manage complexity through abstraction
 - People do not think of a car as a set of tens of thousands of individual parts
 - They think of it as a well-defined object with its own unique behavior
 - They can ignore the details of how the engine, transmission, and braking systems work
 - Instead, they are free to utilize the object as a whole

ENCAPSULATION

- Binds together code and the data it manipulates, and keeps both safe from outside interference and misuse
- Access to the code and data is tightly controlled through a well-defined interface
- Generally the basis of encapsulation is the class

INHERITANCE

- Inheritance is the process by which one object acquires the properties of another object

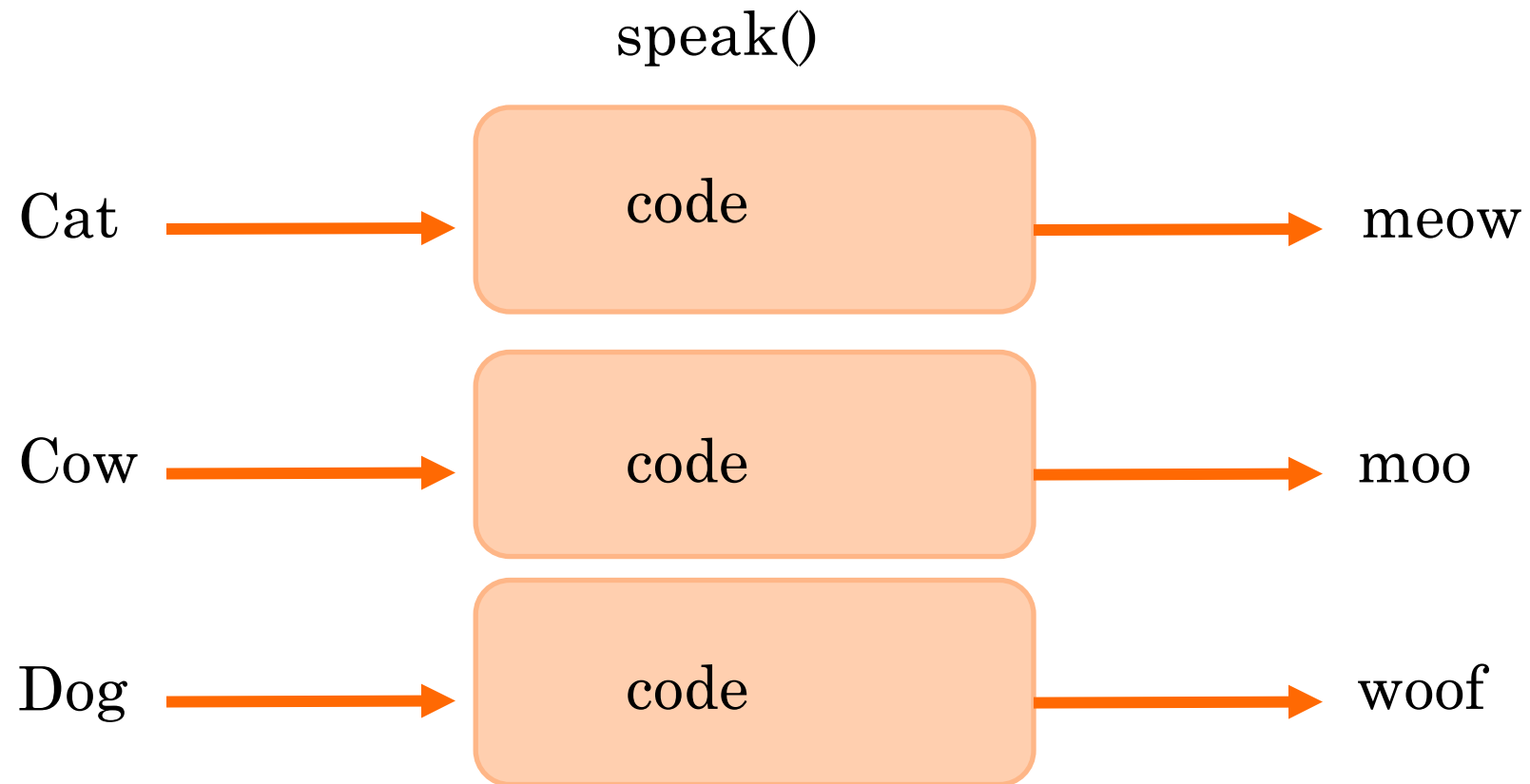


- Without hierarchical classification, each object would need to define all of its characteristics explicitly
- Inheritance helps to eliminate redundant code

POLYMORPHISM

- Poly – Many
- Morph – Form
- Polymorphism is a feature that allows one interface to be used for a general class of actions

POLYMORPHISM



BENEFITS OF OOP

- Abstraction
 - Reduce complexity+ isolate impact of changes
- Encapsulation
 - Reduce complexity+ increase reusability
- Inheritance
 - Eliminate redundant code
- Polymorphism
 - Reduce complexity

SUMMARY

- Procedural Programming
 - List of instructions for a computer to follow
 - Groups of similar tasks are organized as functions
 - Functions share global data
- Object Oriented Programming
 - Emphasis on data rather than procedures/functions
 - Programs are divided into objects
 - Data structures are designed such that they characterize the objects
 - Functions that operate on the data of an object are tied together in the same data structure

INTRODUCTION TO C++

- C++ is the C programmer's answer to Object-Oriented Programming (OOP).
- C++ is an enhanced version of the C language.
- C++ adds support for OOP without sacrificing any of C's power, elegance, or flexibility.
- C++ was invented in 1979 by Bjarne Stroustrup at Bell Laboratories in Murray Hill, New Jersey, USA.

C++ NEW HEADER

- The new-style headers do not have .h extension and do not specify filenames
 - `#include <iostream>`
 - `#include <vector>`
 - `#include <string>`
 - `#include <cstring>`
 - `#include <cmath>`
- Specify standard identifiers that are mapped to files by the compiler
- C++ still support C-style header files, e.g.,
 - `#include <stdio.h>`
 - `#include <string.h>`

C++ NAMESPACE

- A namespace is a declaration region
- It localizes the names of identifiers to avoid name collisions
- The contents of new-style headers are placed in the std namespace
- In order to use the library functions with new-style headers we need to bring std namespace into visibility
 - **using namespace std;**
- In C, the names of the library functions and other such items are not localized by any namespace, instead, they are implicitly placed into global namespace

C++ CONSOLE I/O

- I/O is performed using I/O operator instead of I/O functions
- The output operator (insertion operator) is <<
- The input operator (extraction operator) is >>
- In order to use either << or >> operator programs must begin with the followings

```
#include <iostream>
```

```
...
```

```
using namespace std;
```

C++ CONSOLE I/O (OUTPUT)

- Output operator << is associated with cout, a predefined stream linked to the console output (monitor)
- In general, cout << *expression*

```
printf("Hello World!");  
cout << "Hello World!";
```

```
printf("%d", iCount); /* int iCount */  
cout << iCount;
```

```
printf("%f", 100.99);  
cout << 100.99;
```

```
printf("\n")  
cout << "\n", or cout << '\n', or cout << endl
```

C++ CONSOLE I/O (INPUT)

- Input operator >> is associated with cin, a predefined stream linked to the console input
- In general, cin >> variable;

```
scanf("%s", strName); /* char strName[16] */  
cin >> strName;
```

```
scanf("%d", &iCount); /* int iCount */  
cin >> iCount;
```

```
scanf("%f", &fValue); /* float fValue */  
cin >> fValue;
```

C++ CONSOLE I/O (I/O CHAINING)

- `cout << "Hello" << ' ' << "World" << "!"`;
- `cout << "Value of iCount is: " << iCount`;
- `cout << "Enter day, month, year: "`;
- `cin >> day >> month >> year`;
 - `cin >> day`;
 - `cin >> month`;
 - `cin >> year`

C++ CONSOLE I/O (AN EXAMPLE)

```
#include <iostream>
using namespace std;
int main()
{
    char str[16];
    cout << "Enter a string: ";
    cin >> str;
    cout << "You entered: " << str;
}
```


CLASSES

- C++ Classes are the logical abstraction or model of C++ Objects
- A Class declaration defines a new type
- A class defines the structure and behavior (data and code) that will be shared by a set of objects
 - The data defined by the class are referred to as member variables or instance variables
 - The code that operates on that data is referred to as member methods or just methods
- Classes must be defined before creating the objects, i.e., objects cannot be created without the classes

CLASSES

- General syntax -

```
class class-name
{
    // private functions and variables
public:
    // public functions and variables
} object-list (optional);
```

CLASSES

- Member access specifiers
- public:
 - Can be accessed outside the class directly
 - The public stuff is *the interface*
- private:
 - Accessible only to member functions of class
 - Private members and methods are for internal use only

CLASSES

```
class Rectangle{  
    int height, width;  
public:  
    void set_values (int,int);  
    int area() {return (height*width);}  
};
```

```
void Rectangle::set_values (int a, int b) {  
    height = a;  
    width = b;  
};
```

CLASSES

```
class Circle{
    double radius;
public:
    void setRadius(double r) {radius = r;}
    double getDiameter() { return radius *2;}
    double getArea();
    double getCircumference();
};
double Circle::getArea()
{
    return radius * radius * (22.0/7);
}
double Circle:: getCircumference()
{
    return 2 * radius * (22.0/7);
};
```

C++ OBJECTS

- C++ Classes are used as the type specifier to create C++ Objects

Rectangle recta, rectb;

- An object declaration creates a physical entity of its class type, i.e., occupies memory space class type
- Each object has its own copy of data

C++ OBJECTS

```
int main () {  
  
    Rectangle recta, rectb;  
    recta.set_values (3,4);  
    rectb.set_values (5,6);  
  
    cout << "recta area: " << recta.area() << endl;  
    cout << "rectb area: " << rectb.area() << endl;  
  
    recta.height=5;  
  
    // Not possible, height is a private member  
  
    return 0;  
}
```

FUNCTION OVERLOADING

- Two or more functions can share the same name as long as either
 - The type of their arguments differs, or
 - The number of their arguments differs, or
 - Both of the above
- The compiler will automatically select the correct version
- The return type alone is not a sufficient difference to allow function overloading

FUNCTION OVERLOADING

```
#include <iostream>
using namespace std;

class printData {
public:
    void print(int i) {
        cout << "Printing int: " << i << endl;
    }
    void print(double f) {
        cout << "Printing float: " << f << endl;
    }
    void print(char* c) {
        cout << "Printing character: " << c << endl;
    }
};
```

FUNCTION OVERLOADING

```
int main(void) {  
    printData pd;  
  
    // Call print to print integer  
    pd.print(5);  
  
    // Call print to print float  
    pd.print(500.263);  
  
    // Call print to print character  
    pd.print("Hello C++");  
  
    return 0;  
}
```

Printing int: 5
Printing float: 500.263
Printing character: Hello C++

FUNCTION OVERLOADING

```
int sum (int x, int y)
{
    cout << x+y;
}
```

```
double sum(double x, double y)
{
    cout << x+y;
}
```

FUNCTION OVERLOADING

```
int sum (int x, int y)
{
    cout << x+y;
}
```

```
int sum(int x, int y, int z)
{
    cout << x+y+z;
}
```

FUNCTION OVERLOADING

// This is incorrect and will not compile.

```
int f1 (int a);
```

```
double f1 (int a);
```

.

.

.

```
f1(10)           //which function does the computer call???
```

C++ COMMENTS

- Multi-line comments
 - `/* one or more lines of comments */`
- Single line comments
 - `// ...`

C++ KEYWORDS (PARTIAL LIST)

- bool
- catch
- delete
- false
- friend
- inline
- namespace
- new
- operator
- private
- protected
- public
- template
- this
- throw
- true
- try
- using
- virtual
- wchar_t

SOME DIFFERENCES BETWEEN C AND C++

- No need to use “void” to denote empty parameter list.
- All functions must be prototyped.
- If a function is declared as returning a value, it ***must*** return a value.
- Return type of all functions must be declared explicitly.
- Local variables can be declared anywhere.
- C++ defines the **bool** datatype, and keywords **true** (any nonzero value) and **false** (zero).

QUESTION: WHY CALLED INSERTION AND EXTRACTION

- The output operator (insertion operator) is <<
 - Output operator << is associated with cout, a predefined stream linked to the console output (monitor)
 - Output operator inserts information into a stream
- The input operator (extraction operator) is >>
 - Input operator >> is associated with cin, a predefined stream linked to the console input
 - Input operation removes (i.e., extracts) data from the stream

QUESTION: STRING INPUT

```
#include <iostream>
using namespace std;
int main()
{
    char str[100];
    cout << "Enter a string: ";
    cin >> str;
    cout << "You entered: " << str;
}
```

Enter a string: This is a string
You entered: This

...Program finished with exit code 0
Press ENTER to exit console.

QUESTION: STRING INPUT

```
#include <iostream>
using namespace std;
int main()
{
    char str[100];
    cout << "Enter a string: ";
    cin.get(str, 100);
    cout << "You entered: " << str;
}
```

Enter a string: This is a string

You entered: This is a string

...Program finished with exit code 0

Press ENTER to exit console.

QUESTION: STRING INPUT

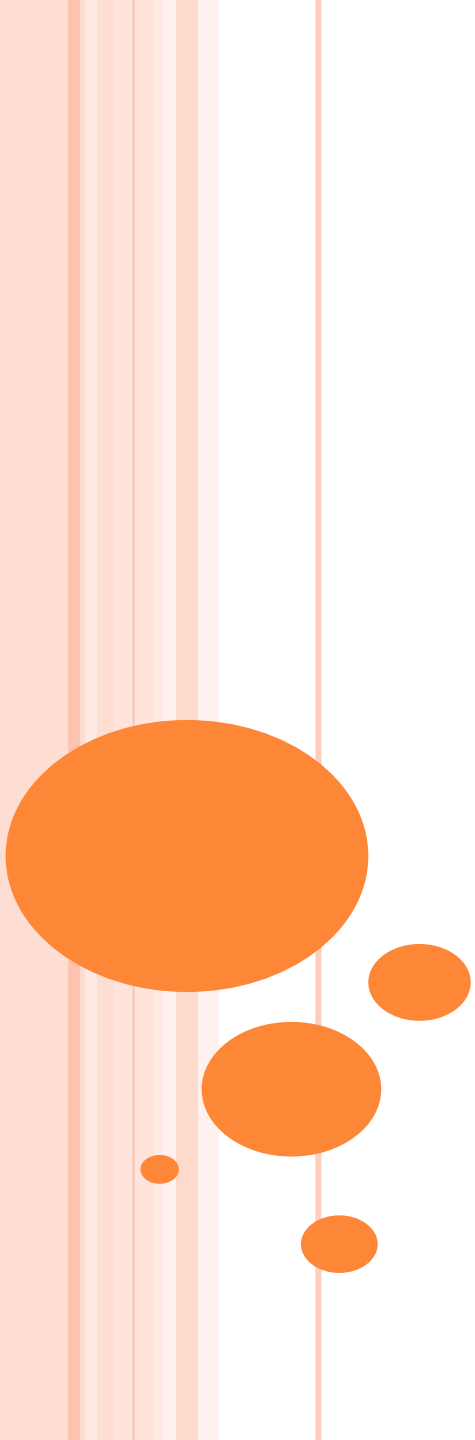
```
#include <iostream>
#include <string>
using namespace std;
int main()
{
    string str;
    cout << "Enter a string: ";
    getline(cin, str);
    cout << "You entered: " << str;
}
```

Enter a string: This is a string

You entered: This is a string

...Program finished with exit code 0

Press ENTER to exit console.



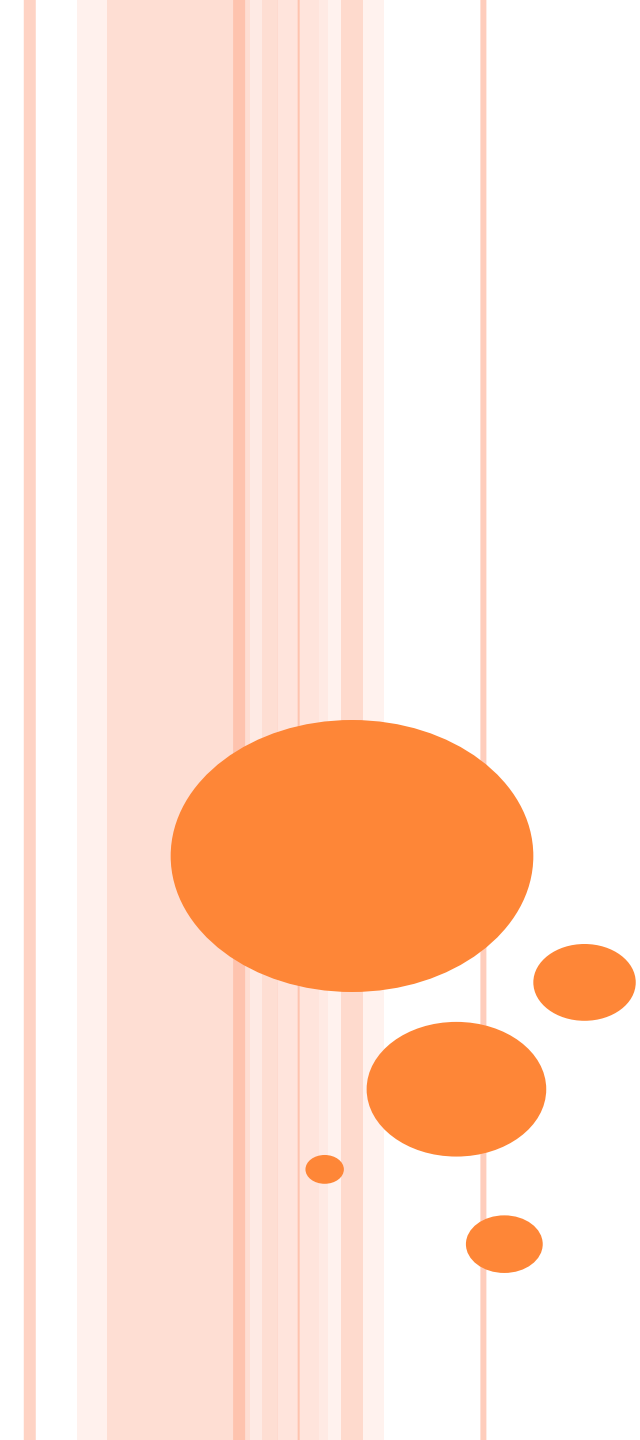
Acknowledgement

<http://faizulbari.buet.ac.bd/Courses.html>

<http://mhkabir.buet.ac.bd/cse201/index.html>

THE END

Topic Covered: Chapter 1



CSE 107: OBJECT ORIENTED PROGRAMMING LANGUAGE

Dr. Tanzima Hashem
Professor
CSE, BUET

FUNCTION OVERLOADING

- Two or more functions can share the same name as long as either
 - The type of their arguments differs, or
 - The number of their arguments differs, or
 - Both of the above
- The compiler will automatically select the correct version
- The return type alone is not a sufficient difference to allow function overloading

FUNCTION OVERLOADING

```
#include <iostream>
using namespace std;

class printData {
public:
    void print(int i) {
        cout << "Printing int: " << i << endl;
    }
    void print(double f) {
        cout << "Printing float: " << f << endl;
    }
    void print(char* c) {
        cout << "Printing character: " << c << endl;
    }
};
```


FUNCTION OVERLOADING

```
int main(void) {  
    printData pd;  
  
    // Call print to print integer  
    pd.print(5);  
  
    // Call print to print float  
    pd.print(500.263);  
  
    // Call print to print character  
    pd.print("Hello C++");  
  
    return 0;  
}
```

```
Printing int: 5  
Printing float: 500.263  
Printing character: Hello C++
```

FUNCTION OVERLOADING

```
int sum (int x, int y)
{
    cout << x+y;
}
```

```
double sum(double x, double y)
{
    cout << x+y;
}
```

FUNCTION OVERLOADING

```
int sum (int x, int y)
{
    cout << x+y;
}
```

```
int sum(int x, int y, int z)
{
    cout << x+y+z;
}
```

FUNCTION OVERLOADING

// This is incorrect and will not compile.

```
int f1 (int a);
```

```
double f1 (int a);
```

•

•

•

```
f1(10)           //which function does the computer call???
```

OVERLOADING CONSTRUCTOR FUNCTIONS

- It is possible to overload constructors, but destructors cannot be overloaded
- Three main reasons to overload a constructor function
 - To gain flexibility
 - To support arrays
 - To create copy constructors

OVERLOADING CONSTRUCTOR FUNCTIONS

- Overloading constructor functions also allows the programmer to select the most convenient method to create objects
 - **Date d1(22, 9, 2007);** // uses Date(int d, int m, int y)
 - **Date d2("22-Sep-2007");** // uses Date(char* str)
- There must be a constructor function for each way that an object of a class will be created, otherwise compile-time error occurs
- Let, we want to write
 - **MyClass ob1, ob2(10);**
- Then MyClass should have the following two constructors (it may have more)
 - **MyClass () { ... }**
 - **MyClass (int n) { ... }**
- Whenever we write a constructor in a class, the compiler does not supply the default no argument constructor automatically

OVERLOADING CONSTRUCTOR FUNCTIONS

- No argument constructor is also necessary for declaring arrays of objects without any initialization
 - **MyClass array1[5];**
// uses MyClass () { ... } for each element
- But with the help of an overloaded constructor, we can also initialize the elements of an array while declaring it
 - **MyClass array2[3] = {1, 2, 3}**
// uses MyClass (int n) { ... } for each element
- As dynamic arrays of objects cannot be initialized, the class must have a no argument constructor to avoid compiler error while creating dynamic arrays using “new”.

COPY CONSTRUCTOR

- Copy constructor is a constructor which is a constructor which creates an object by initializing it with an object of the same class

The copy constructor is used to:

- Initialize one object from another object of the same type in a declaration statement

```
MyClass y;  
MyClass x = y;
```

- Copy an object to pass it as an argument to a function

```
func1(y); // calls “void func1( MyClass obj )”
```

- Copy an object to return it from a function

```
y = func2(); // gets the object returned from “MyClass func2()”
```


COPY CONSTRUCTOR

- If we do not write our own copy constructor, then the compiler supplies a copy constructor that simply performs bitwise copy
- If the class has pointer variables and dynamic memory allocations then bitwise copy is not enough
- We can write our own copy constructor to dictate precisely how members of one object should be copied to other
- The most common form of copy constructor is

```
classname (const classname &obj) {  
    // body of constructor  
}
```

COPY CONSTRUCTOR

```
#include <iostream>
using namespace std;

class Rectangle {
    int *width, *height;
public:
    Rectangle(int, int);
    ~Rectangle ();
    int area () {return (*width * *height);}
};

Rectangle::Rectangle (int a, int b) {
    width= new int;
    height = new int;
    *width = a;
    *height = b;
}
```

COPY CONSTRUCTOR

```
Rectangle::~Rectangle () {  
    delete width;  
    delete height;  
}
```

```
Rectangle larger(Rectangle recta, Rectangle rectb){  
    if(recta.area()>rectb.area())  
        return recta;  
    else  
        return rectb;  
}
```

COPY CONSTRUCTOR

```
int main () {  
    Rectangle recta (3,4);  
    Rectangle rectb (5,6);  
  
    Rectangle rectc=recta; //this will cause the program to crash  
  
    Rectangle rect_larger(0,0);  
  
    rect_larger=larger(recta, rectb); //this will cause the program to crash  
  
    cout << "recta area: " << recta.area() << endl;  
    cout << "rectb area: " << rectb.area() << endl;  
    cout << "rectc area: " << rectc.area() << endl;  
  
    cout << "rect_larger area: " << rect_larger.area() << endl;  
  
    return 0;  
}
```

COPY CONSTRUCTOR

```
#include <iostream>
using namespace std;

class Rectangle {
    int *width, *height;
public:
    Rectangle(int, int);
    Rectangle(const Rectangle &r);
    ~Rectangle ();
    int area () {return (*width * *height);}
};

Rectangle::Rectangle (int a, int b) {
    width= new int;
    height = new int;
    *width = a;
    *height = b;
}
```

COPY CONSTRUCTOR

```
Rectangle::Rectangle(const Rectangle &r){  
    width=new int;  
    height=new int;  
    *width=*r.width;  
    *height=*r.height;  
}
```

```
Rectangle::~~Rectangle () {  
    delete width;  
    delete height;  
}
```

```
Rectangle larger(Rectangle recta, Rectangle rectb){  
    if(recta.area()>rectb.area())  
        return recta;  
    else  
        return rectb;  
}
```

COPY CONSTRUCTOR

```
int main () {  
    Rectangle recta (3,4);  
    Rectangle rectb (5,6);  
  
    Rectangle rectc=recta; //this will call copy constructor  
    Rectangle rect_larger(0,0);  
  
    rect_larger=larger(recta, rectb);  
    //this will call both copy constructor and destructor 3 times  
  
    cout << "recta area: " << recta.area() << endl;  
    cout << "rectb area: " << rectb.area() << endl;  
    cout << "rectc area: " << rectc.area() << endl;  
  
    cout << "rect_larger area: " << rect_larger.area() << endl;  
  
    return 0;  
}
```

USING DEFAULT ARGUMENTS

- Related to function overloading
 - Essentially a shorthand form of function overloading
- Allows to give a parameter a default value when no corresponding argument is specified when the function is called
 - **`void f1(int a = 0, int b = 0) { ... }`**
 - It can now be called in three different ways
 - **`f1();`** // inside `f1()` 'a' is '0' and b is '0'
 - **`f1(10);`** // inside `f1()` 'a' is '10' and b is '0'
 - **`f1(10, 99);`** // inside `f1()` 'a' is '10' and b is '99'
 - We cannot give 'b' a new (non-default) value without specifying a new value for 'a'
 - While specifying non-default values, we have to start from the leftmost parameter and move to the right one by one

USING DEFAULT ARGUMENTS

- Default arguments must be specified only once: either in the function's prototype or in its definition
- All default parameters must be to the right of any parameters that don't have defaults
 - **`void f2(int a, int b = 0); // no problem`**
 - **`void f3(int a, int b = 0, int c = 5); // no problem`**
 - **`void f4(int a = 1, int b); // compiler error`**
- Default arguments must be constants or global variables.
- Default arguments cannot be local variables or other parameters

USING DEFAULT ARGUMENTS

- Relation between default arguments and function overloading
 - **void f1(int a = 0, int b = 0) { ... }**
 - It acts as the same way as the following overloaded functions –
 - **void f2() { int a = 0, b = 0; ... }**
 - **void f2(int a) { int b = 0; ... }**
 - **void f2(int a, int b) { ... }**
- Constructor functions can also have default arguments
- It is possible to create copy constructors that take additional arguments, as long as the additional arguments have default values
 - **MyClass(const MyClass &obj, int x = 0) { ... }**

OVERLOADING AND AMBIGUITY

- Due to automatic type conversion rules
- Example 1:
 - `void f1( float f ) { ... }`
 - `void f1( double d ) { ... }`
 - `float x = 10.09;`
 - `double y = 10.09;`
 - `f1(x);` // unambiguous – use `f1(float)`
 - `f1(y);` // unambiguous – use `f1(double)`
 - `f1(10);` // ambiguous, compiler error
 - Because integer ‘10’ can be promoted to both “float” and “double”.

OVERLOADING AND AMBIGUITY

- Due to the use of reference parameters
- Example 2:
 - `void f2( int a, int b ) { ... }`
 - `void f2(int a, int &b ) { ... }`
 - `int x = 1, y = 2;`
 - `f2(x, y); // ambiguous, compiler error`

OVERLOADING AND AMBIGUITY

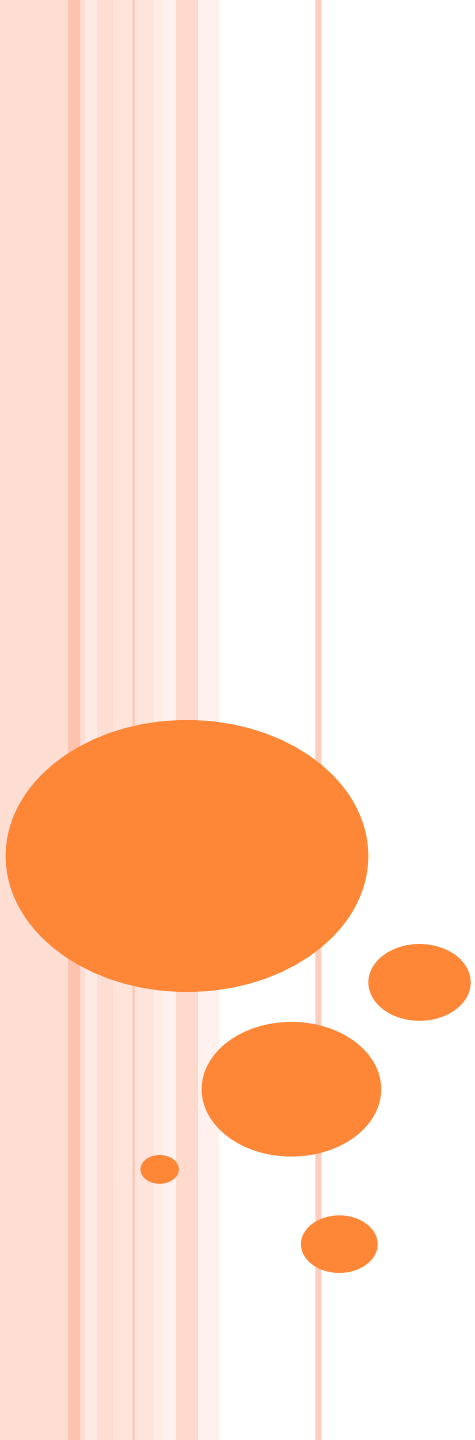
- Due to the use of default arguments
- Example 3:
 - `void f3( int a ) { ... }`
 - `void f3(int a, int b = 0 ) { ... }`
 - `f3(10, 20);` // unambiguous – calls `f3(int, int)`
 - `f3(10);` // ambiguous, compiler error

FINDING THE ADDRESS OF AN OVERLOADED FUNCTION

○ Example:

- `void space( int a ) { ... }`
- `void space( int a, char c ) { ... }`
- `void (*fp1)(int);`
- `void (*fp2)(int, char);`
- `fp1 = space; // gets address of space(int)`
- `fp2 = space; // gets address of space(int, char)`

- So, it is the declaration of the pointer that determines which function's address is assigned



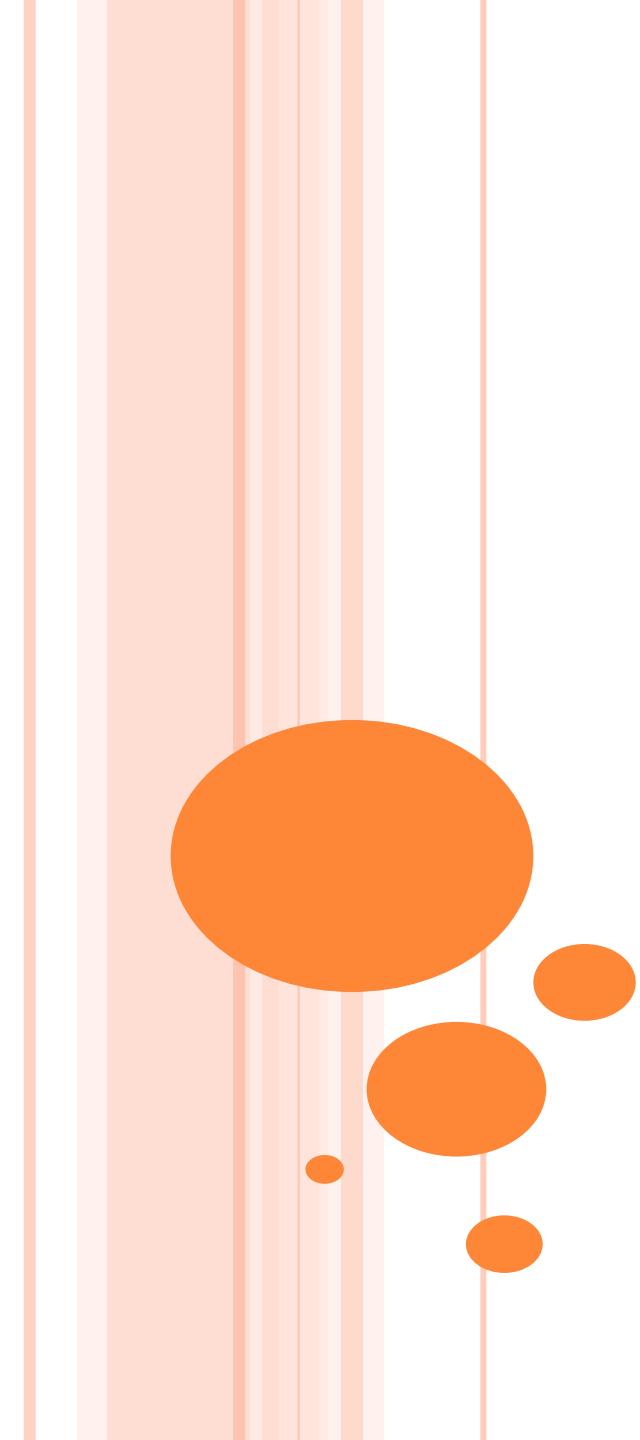
Acknowledgement

<http://faizulbari.buet.ac.bd/Courses.html>

<http://mhkabir.buet.ac.bd/cse201/index.html>

THE END

Topic Covered: Chapter 5 (except 5.3)



CSE 107: OBJECT ORIENTED PROGRAMMING LANGUAGE

Dr. Tanzima Hashem
Professor
CSE, BUET

OPERATOR OVERLOADING

- Operator overloading is a type of function overloading
- An operator is always overloaded relative to a class (an user defined data type)
- An overloaded operator gets a special meaning relative to its class
However, the operator does not loose its original meaning relative to other data types
- To overload an operator an operator function is defined for the class
- The operator function can be a member or a friend function of the class

OPERATOR OVERLOADING RESTRICTIONS

- You cannot overload
 - :: (scope resolution)
 - . (member selection)
 - .* (member selection through pointer to function)
 - ? (ternary conditional) operators
- Overloading cannot change the original precedence of the operator
- The number of operands on which the operator would be applicable cannot be changed too
- Operator functions cannot have default arguments

OPERATOR OVERLOADING GENERAL FORM

- Prototype definition

```
class class-name{  
    .....  
    return-type operator # (arg-list);  
};
```

- Function definition

```
return-type class-name :: operator # (arg-list){  
    //operation to be performed  
}
```

OVERLOADING BINARY OPERATORS

```
class coord {  
    int x, y;  
public:  
    coord(int a = 0, int b = 0) {  
        x = a; y = b;  
    }  
    void show() {  
        cout << x << ", " << y << endl;  
    }  
    coord operator+(coord obj);  
    coord operator+(int i);  
    coord operator-(coord obj);  
    coord operator=(coord obj);  
};
```

OVERLOADING BINARY OPERATORS

```
coord coord::operator+(coord obj) {  
    coord temp;  
    temp.x = x + obj.x;  
    temp.y = y + obj.y;  
    return temp;  
}  
  
coord coord::operator+(int i) {  
    coord temp;  
    temp.x = x + i;  
    temp.y = y + i;  
    return temp;  
}
```

OVERLOADING BINARY OPERATORS

```
coord coord::operator-(coord obj) {  
    coord temp;  
    temp.x = x - obj.x;  
    temp.y = y - obj.y;  
    return temp;  
}
```

```
coord coord::operator=(coord obj) {  
    x = obj.x;  
    y = obj.y;  
    return *this;  
}
```

OVERLOADING BINARY OPERATORS

```
void main() {  
    coord c1(20, 20), c2(10, 10);  
    coord c3 = c1 + c2; // c1.+(c2)  
    c3.show(); // 30, 30  
  
    coord c4 = c3 + 5; // c3.+(5)  
    c4.show(); // 35, 35  
  
    coord c5 = c2 - c1; // c2.-(c1)  
    c5.show(); // -10, -10
```

```
    coord c6 = c1 + c2 + c3;  
    // (c1.+(c2)).+(c3)  
    c6.show(); // 60, 60  
    (c6 - c4).show(); // 25, 25  
  
    c5 = c6 = c6 - c1;  
    // c5.=(c6.=(c6.-(c1)))  
    c5.show(); // 40, 40  
    c6.show(); // 40, 40  
}
```

OVERLOADING THE RELATIONAL AND LOGICAL OPERATORS

```
class coord {  
    int x, y;  
public:  
    coord(int a = 0, int b = 0) {  
        x = a; y = b;  
    }  
    void show() {  
        cout << x << ", " << y << endl;  
    }  
    int operator==(coord obj);  
    int operator!=(coord obj);  
    int operator&&(coord obj);  
    int operator || (coord obj);  
};
```


OVERLOADING THE RELATIONAL AND LOGICAL OPERATORS

```
int coord::operator==(coord obj) {  
    return (x == obj.x) && (y == obj.y);  
}  
int coord::operator!=(coord obj) {  
    return (x != obj.x) || (y != obj.y);  
}  
int coord::operator&&(coord obj) {  
    return (x && obj.x) && (y && obj.y);  
}  
int coord::operator|| (coord obj) {  
    return (x || obj.x) || (y || obj.y);  
}
```

OVERLOADING A UNARY OPERATOR

```
class coord {  
    int x, y;  
public:  
    coord(int a = 0, int b = 0) {  
        x = a; y = b;  
    }  
    void show() {  
        cout << x << ", " << y << endl;  
    }  
    coord operator++();  
    coord operator-();  
    coord operator-(coord obj);  
};
```

OVERLOADING A UNARY OPERATOR

```
coord coord::operator++() {  
    ++x; ++y; return *this;  
} // prefix version
```

```
coord coord::operator-() {  
    coord temp;  
    temp.x = -x; temp.y = -y;  
    return temp;  
}
```

```
coord coord::operator-(coord obj) {  
    coord temp;  
    temp.x = x-obj.x; temp.y = y-obj.y;  
    return temp;  
}
```

OVERLOADING A UNARY OPERATOR

```
void main() {  
    coord c1(10, 10), c2(10, 10);  
    coord c3 = ++c1;  
        // c1.++()  
    c1.show(); // 11, 11  
    c2.show(); // 10, 10  
    c3.show(); // 11, 11  
  
    coord c5 = -c1;  
        // c1.-()  
    c1.show(); // 11, 11  
    c5.show(); // -11, -11
```

```
coord c6 = c3 - c2;  
        // c3.-(c2)  
    c6.show(); // 1, 1  
}
```

- Postfix increment
- coord operator++(int unused);

OBJECT COPY ISSUES

- Whenever possible we should use reference parameters while passing objects to or returning objects from a function
 - `coord coord::operator+(coord& obj) { ... }`
 - `coord& coord::operator=(coord& obj) { ... }`
 - `coord& coord::operator++() { ... }`
- Otherwise should use copy constructors to overcome object copy problems

USING FRIEND OPERATOR FUNCTIONS

- It is possible to overload an operator relative to a class by using a friend rather than a member function
- As a friend function does not have a *this* pointer –
 - For binary operators, both operands must be passed explicitly
 - For unary operators, the single operand must be passed explicitly
- Allows us to perform operations like -
 - `coord c1(10, 10), c2;`
 - `c2 = 10 + c1;`
 - We cannot perform this using member operator functions as the left argument of '+' is not an object of class "coord"
- We cannot use a friend to overload the assignment operator (=)
 - It can be overloaded only by a member operator function

USING FRIEND OPERATOR FUNCTIONS

```
class coord {  
    int x, y;  
public:  
    coord(int a = 0, int b = 0) {  
        x = a; y = b;  
    }  
    void show() {  
        cout << x << ", " << y << endl;  
    }  
    friend coord operator+(coord ob1, coord ob2);  
    friend coord operator+(int i, coord ob);  
    friend coord operator++(coord &ob);  
};
```

USING FRIEND OPERATOR FUNCTIONS

```
coord operator+(coord ob1, coord ob2) {  
    coord temp;  
    temp.x = ob1.x + ob2.x;  
    temp.y = ob1.y + ob2.y;  
    return temp;  
}
```

```
coord operator+(int i, coord ob) {  
    coord temp;  
    temp.x = ob.x + i;  
    temp.y = ob.y + i;  
    return temp;  
}
```


USING FRIEND OPERATOR FUNCTIONS

```
coord operator++(coord & ob) {  
    ob.x++;  
    ob.y++;  
    return ob;  
}
```

- Here, in case of “++” we must use reference parameter
- Otherwise changes made inside the function will not be visible outside and the original object will remain unchanged

USING FRIEND OPERATOR FUNCTIONS

```
void main() {  
    coord c1(20, 20), c2(10, 10);  
    coord c3 = c1 + c2;  
    // +(c1, c2)  
    c3.show(); // 30, 30  
    coord c4 = 5 + c3;  
    // +(5, c3)  
    c4.show(); // 35, 35  
    ++c4;  
    // ++(c4)  
    c4.show(); // 36, 36  
}
```

A CLOSER LOOK AT THE ASSIGNMENT OPERATOR

- By default, “ob1 = ob2” places a bitwise copy of “ob2” into “ob1”
- This causes problem when class members point to dynamically allocated memory
- Copy constructor is of no use in this case as it is an *assignment*, not an initialization
- So, we need to overload ‘=’ to overcome such problems

A CLOSER LOOK AT THE ASSIGNMENT OPERATOR

```
class strtype {  
    char *p;  
    int len;  
public:  
    strtype(char *s) {  
        len = strlen(s) + 1;  
        p = new char[len];  
        strcpy(p, s);  
    }  
    ~strtype() {  
        delete [ ] p;  
    }  
    strtype &operator=(strtype &ob);  
};
```

A CLOSER LOOK AT THE ASSIGNMENT OPERATOR

```
strtype &strtype::operator=(strtype &ob) {  
    if(len < ob.len) {  
        delete [ ] p;  
        p = new char[ob.len];  
    }  
    len = ob.len;  
    strcpy(p, ob.p);  
    return *this;  
}  
void main() {  
    strtype s1("BUET"), s2("CSE");  
    s1 = s2; // no problem  
}
```

A CLOSER LOOK AT THE ASSIGNMENT OPERATOR

- The overloaded '=' operator must return *this to allow chains of assignments
 - `ob1 = ob2 = ob3 = ob4;`
- If the overloaded '=' operator returns nothing (void) then
 - `ob1 = ob2;` is possible, but
 - `ob1 = ob2 = ob3;` produces compiler error
 - `ob3` can be assigned to `ob2`, but then it becomes "`ob1 = (void)`"
 - So, the compiler detects it early and flags it as an error
- Whenever possible we should use references while passing objects to functions
 - Copy constructors can also help in this regard but using references is more efficient as no copy is performed

A CLOSER LOOK AT THE ASSIGNMENT OPERATOR

- Overloading the '=' operator, we can assign object of one class to an object of another class

```
class yourclass { ... };  
class myclass {  
public:  
    myclass& operator=(yourclass &obj) {  
        // assignment activities  
        return *this;  
    };  
};
```

```
void main( ) {  
    myclass m1, m2;  
    yourclass y;  
    m1 = y;  
    m2 = m1;  
}
```

OVERLOADING THE [] SUBSCRIPT OPERATOR

- In C++, the [] is considered a binary operator for the purposes of overloading
- The [] can be overloaded only by a member function
- General syntax
 - **ret-type class-name::operator[](int index) {...}**
 - “index” does not have to be of type “int”
 - “index” can be of any other type
 - **ret-type class-name::operator[](char *index) {...}**
- It is useful when the class has some array like behavior

OVERLOADING THE [] SUBSCRIPT OPERATOR (EXAMPLE -1)

```
class array {  
    int a[3];  
public:  
    array() {  
        for(int i=0; i<3; i++)  
            a[i] = i;  
    }  
    int operator[ ](int i) {  
        return a[i];  
    }  
    int operator[ ](char *s);  
};  
int array::operator[ ](char *s) {  
    if(strcmp(s, "zero")==0)  
        return a[0];
```

```
    else if(strcmp(s, "one")==0)  
        return a[1];  
    else if(strcmp(s, "two")==0)  
        return a[2];  
    return -1;  
}  
void main() {  
    array ob;  
    cout << ob[1]; // 1  
    cout << ob["two"]; // 2  
    ob[0] = 5; // compiler error  
    // ob[i] is not an l-value in this example  
}
```

OVERLOADING THE [] SUBSCRIPT OPERATOR (EXAMPLE -2)

```
class array {  
    int a[3];  
public:  
    array() {  
        for(int i=0; i<3; i++)  
            a[i] = i;  
    }  
    int& operator[ ](int i) {  
        return a[i];  
    }  
    int& operator[ ](char *s);  
};  
int& array::operator[ ](char *s) {  
    if(strcmp(s, "zero")==0)  
        return a[0];
```

```
    else if(strcmp(s, "one")==0)  
        return a[1];  
    else if(strcmp(s, "two")==0)  
        return a[2];  
    return a[0];  
}  
void main() {  
    array ob;  
    cout << ob[1]; // 1  
    cout << ob["two"]; // 2  
    ob[0] = 5; // no problem  
    // ob[i] is now both an l-value and r-value  
    cout << ob["zero"]; // 5
```

```
}
```

NOTE ON L-VALUE

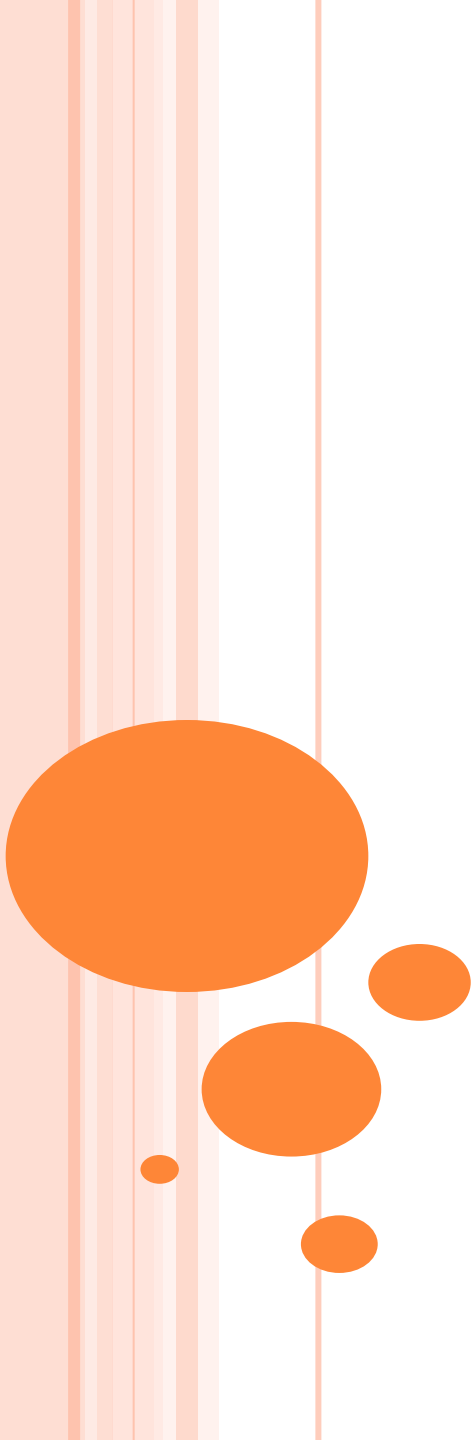
- An l-value is an expression that can appear on both the left-hand and right-hand side of an assignment
- It represents a location not just a value
- Based on its placement, either the location or the value is used by the compiler
 - `int x, y; x = 0; y = x;`
 - Here both `x` and `y` are l-values
- Generally if a function returns a value of any type, then it cannot be used as an l-value
 - `int f1( ) { int x = 0; return x; }`
 - `int n = f1( ); // no problem`
 - `f1( ) = n; // compiler error, need a location to place a value`

NOTE ON L-VALUE

- But if a function returns a *reference* of any type, then it can be used both as an l-value and r-value
 - `int x; // global variable`
 - `int& f1( ) { return x; }`
 - `int n = f1( ); // no problem`
 - Works like “`int n = x`”
 - `f1( ) = n; // no problem`
 - Works like “`x = n`”
- Data can be both fetched from and written into an l-value

NOTE ON R-VALUE

- If an expression is just an ***r-value*** then it cannot appear on the left-hand side of an assignment
- It represents just a value
 - `int x = 3; // x is both an l-value and r-value`
 - `3 = x; // compiler error, 3 is just an r-value, not an l-value`
- Generally if a function returns a value of any type, then it can only be used as an r-value
 - `int f1( ) { int x = 2; return x; }`
 - `int n = f1( ); // no problem`
 - `cout << f1( ); // no problem, prints '2'`
 - `f1( ) = n; // compiler error`



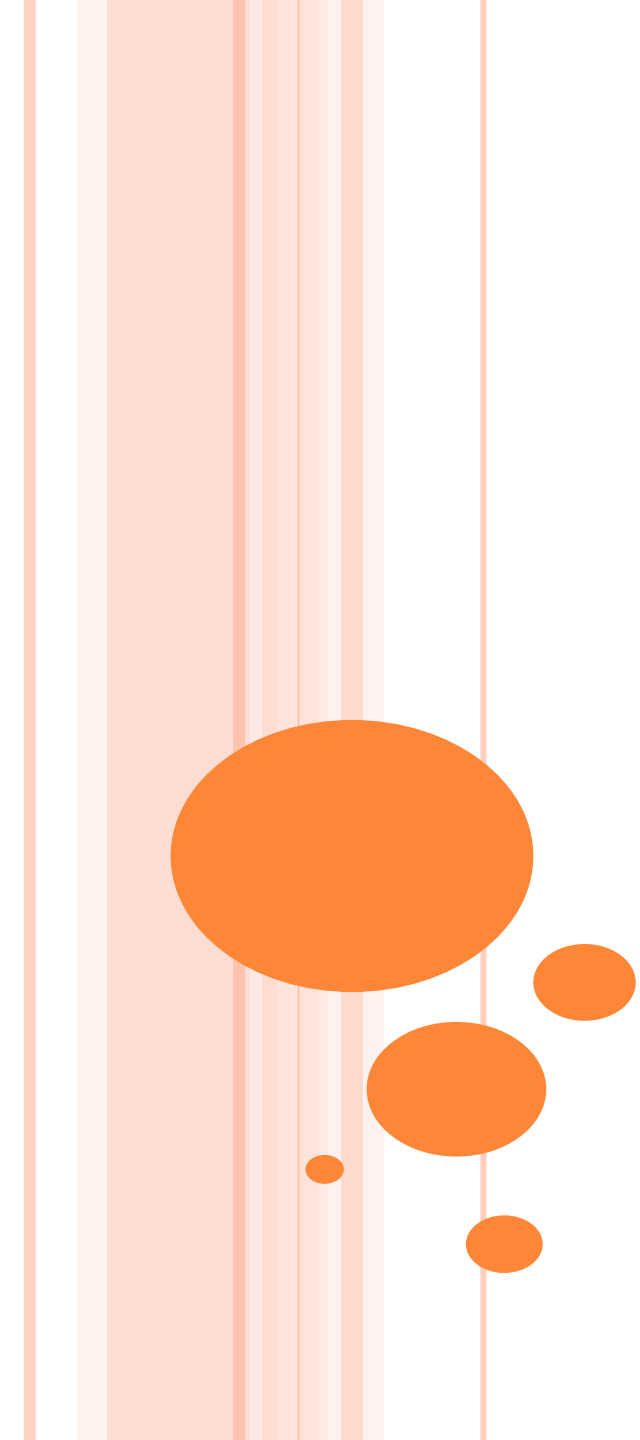
Acknowledgement

<http://faizulbari.buet.ac.bd/Courses.html>

<http://mhkabir.buet.ac.bd/cse201/index.html>

THE END

Topic Covered: Chapter 6



CSE 107: OBJECT ORIENTED PROGRAMMING LANGUAGE

Dr. Tanzima Hashem
Professor
CSE, BUET

INHERITANCE

- Inheritance allows one object to inherit member variables and/or member functions from another object
- Inherited object is called base object
- Inheriting object is called derived object
- Helps programmer to write reusable code
- Helps programmer to write compact code

INHERITANCE

- Inheritance starts with defining the base class first

```
class base-class-name {
```

```
.....
```

```
};
```

- Derived class is then defined using the base class
- The general form of deriving a class from another class is as follows:

```
class derived-class-name: access base-class-name {
```

```
.....
```

```
};
```

- Access can be either private or public or protected
- Default access is private for derived class, public for derived structure

VISIBILITY OF BASE CLASS MEMBERS IN DERIVED CLASS

Member access specifier in base class	Member visibility in derived class		
	Type of Inheritance		
	Private	Protected	Public
Private			
Protected			
Public			

VISIBILITY OF BASE CLASS MEMBERS IN DERIVED CLASS

Member access specifier in base class	Member visibility in derived class		
	Type of Inheritance		
	Private	Protected	Public
Private	Not Inherited	Not Inherited	Not Inherited
Protected	Private	Protected	Protected
Public	Private	Protected	Public

INHERITANCE: BASE CLASS

```
#include <iostream>
using namespace std;
class Rectangle {
    int width, length;
public:
    void set_width(int w) {width=w;}
    void set_length(int l) {length=l;}
    int area() {return (width*length);}
};
```

INHERITANCE: DERIVED CLASS

```
class Box: public Rectangle {  
    int height;  
public:  
    void set_height (int h){height=h;}  
    int volume () {return (area()*height);}  
    /*private members of the base class cannot be accessed by the derived  
    class*/  
    //int volume () {return (width*length*height);}  
};
```

INHERITANCE:

```
int main(){
    Rectangle rect;
    Box box;

    rect.set_width(3);
    rect.set_length(4);

    box.set_width(4); //inherited
    box.set_length(4); //inherited
    box.set_height(5);

    cout<<"Rectangle area: "<<rect.area()<<endl;
    cout<< "Box base area: base area: "<<box.area()<<endl; //inherited
    cout<<"Box volume: "<<box.volume()<<endl;
    return 0;
}
```

INHERITANCE: DERIVED CLASS

```
class Box: private Rectangle {  
    int height;  
public:  
    void set_height (int h){height=h;}  
    int volume () {return (area()*height);}  
};
```

INHERITANCE

```
int main(){
    Rectangle rect;
    Box box;

    rect.set_width(3);
    rect.set_length(4);

    //box.set width(4);
    //box.set_length(4);
    box.set_height(5);

    cout<<"Rectangle area: "<<rect.area()<<endl;
    /* function area() is private to Box object cannot be accessed outside Box object */
    //cout<< "Box base area: base area: "<<box.area()<<endl;
    cout<<"Box volume: "<<box.volume()<<endl;
    return 0;
}
```


INHERITANCE: BASE CLASS

```
#include <iostream>
using namespace std;
class Rectangle {
protected:
    int width, length;
public:
    void set_width(int w) {width=w;}
    void set_length(int l) {length=l;}
    int area() {return (width*length);}
};
```

INHERITANCE: DERIVED CLASS

```
class Box: public Rectangle {  
    int height;
```

```
public:
```

```
    void set_height (int h){height=h;}
```

```
/*protected members of the base class can be accessed by the derived class */
```

```
    int volume () {return (width*length*height);}
```

```
};
```

INHERITANCE

```
int main(){
    Rectangle rect;
    Box box;

    //rect.width=4; error!
    rect.set_width(3);
    rect.set_length(4);

    //box.width=4; error!
    box.set_width(4); //inherited
    box.set_length(4); //inherited
    box.set_height(5);

    cout<<"Rectangle area: "<<rect.area()<<endl;
    cout<< "Box base area: "<<box.area()<<endl; //inherited
    cout<<"Box volume: "<<box.volume()<<endl;
    return 0;
}
```

CONSTRUCTORS, DESTRUCTORS, AND INHERITANCE

- Both base class and derived class can have constructors and destructors
- Constructor functions are executed in the order of derivation
- Destructor functions are executed in the reverse order of derivation
- While working with an object of a derived class, the base class constructor and destructor are always executed no matter how the inheritance was done (private, protected or public)

CONSTRUCTORS, DESTRUCTORS, AND INHERITANCE

```
class base {  
public:  
    base() {  
        cout << "Constructing base class\n";  
    }  
    ~base() {  
        cout << "Destructing base class\n";  
    }  
};  
class derived : public base {  
public:  
    derived() {  
        cout << "Constructing derived class\n";  
    }  
    ~derived() {  
        cout << "Destructing derived class\n";  
    }  
};
```

```
void main() {  
    derived obj;  
}
```

Output:

Constructing base class
Constructing derived class
Destructing derived class
Destructing base class

CONSTRUCTORS, DESTRUCTORS, AND INHERITANCE

- If a base class constructor takes parameters then it is the responsibility of the derived class constructor(s) to collect them and pass them to the base class constructor using the following syntax -
 - **derived-constructor(arg-list) : base(arg-list) { ... }**
 - Here “base” is the name of the base class
- It is permissible for both the derived class and the base class to use the same argument
- It is also possible for the derived class to ignore all arguments and just pass them along the base class.

CONSTRUCTORS, DESTRUCTORS, AND INHERITANCE

```
#include <iostream>
using namespace std;

class Rectangle {
    int *width, *length;
public:
    Rectangle();
    Rectangle(int, int);
    ~Rectangle ();
    int area () {return (*width * *length);}
};

Rectangle::Rectangle () {
    width= new int;
    length = new int;
    *width = 0;
    *length = 0;
}
```

CONSTRUCTORS, DESTRUCTORS, AND INHERITANCE

```
Rectangle::Rectangle (int a, int b) {  
    width= new int;  
    length = new int;  
    *width = a;  
    *length = b;  
}
```

```
Rectangle::~~Rectangle () {  
    delete width;  
    delete length;  
}
```


CONSTRUCTORS, DESTRUCTORS, AND INHERITANCE

```
class Box: public Rectangle{
    int *height;
public:
    Box();
    Box(int, int, int);
    ~Box();
    int volume(){ return area()*(*height);}
};
```

```
Box::Box ()
{
    height=new int;
    *height=0;
}
```

CONSTRUCTORS, DESTRUCTORS, AND INHERITANCE

```
Box::Box (int w, int l, int h):Rectangle(w,l) {  
    height=new int;  
    *height=h;  
}
```

```
Box::~~Box () {  
    delete height;  
}
```

CONSTRUCTORS, DESTRUCTORS, AND INHERITANCE

```
int main(){
    Rectangle rect(3,4);
    Box box (3,4,5);

    cout<<"Rectangle area: "<<rect.area()<<endl;
    cout<< "Box base area: base area:"<<box.area()<<endl;
    cout<<"Box volume: "<<box.volume()<<endl;

    return 0;
}
```

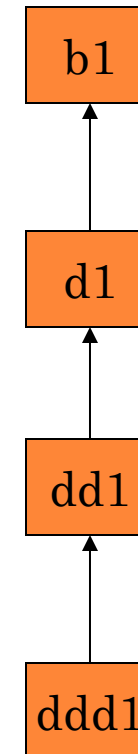
CONSTRUCTORS, DESTRUCTORS, AND INHERITANCE

```
class MyBase {  
public:  
    int x;  
    MyBase(int m) { x = m; }  
};  
class MyDerived : public MyBase {  
public:  
    int y;  
    MyDerived() : MyBase(0) { y = 0; }  
    MyDerived(int a) : MyBase(a)  
    {  
        y = 0;  
    }  
    MyDerived(int a, int b) : MyBase(a)  
    {  
        y = b;  
    }  
};
```

```
void main() {  
    MyDerived o1; // x = 0, y = 0  
    MyDerived o2(5); // x = 5, y = 0  
    MyDerived o3(6, 7); // x = 6, y = 7  
}
```

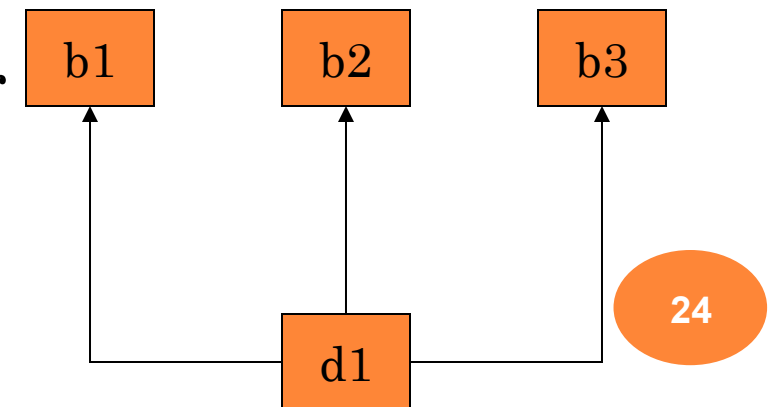
MULTIPLE INHERITANCE: MULTILEVEL CLASS HIERARCHY

- A derived class can inherit more than one base class in two ways
 - Multilevel Class Hierarchy
 - Multiple Direct Inheritance
- Multilevel Class Hierarchy
 - By a chain of inheritance
 - b1 -> d1 -> dd1 -> ddd1 -> ...
 - Here b1 is an indirect base class of both dd1 and ddd1
 - Constructors are executed in the order of inheritance
 - Destructors are executed in the reverse order



MULTIPLE INHERITANCE: MULTIPLE DIRECT INHERITANCE

- A derived class can inherit more than one base class in two ways
 - Multilevel Class Hierarchy
 - Multiple Direct Inheritance
- Multiple Direct Inheritance
 - By directly inheriting more than one base class
 - `class d1 : access b1, access b2, ..., access bN { ... }`
 - Constructors are executed in the order, left to right, that the base classes are specified
 - Destructors are executed in the reverse order



MULTILEVEL CLASS HIERARCHY

```
class Point{  
    double x;  
    double y;  
public:  
    Point(double x, double y){  
        this->x=x;  
        this->y=y;  
    }  
    void get_xy(double &x, double &y){  
        x=this->x;  
        y=this->y;  
    }  
}
```

MULTILEVEL CLASS HIERARCHY

```
class Circle: public Point{
protected:
    double rad;
public:
    Circle(double x, double y, double r);
    double area(){return 3.14*rad*rad;}
}

Circle::Circle(double x, double y, double r):Point(x,y){
    rad=r;
}
```


MULTILEVEL CLASS HIERARCHY

```
class Cylinder: public Circle{  
    double height;
```

```
public:
```

```
    Cylinder(double x, double y, double r, double h);  
    double volume(){return 3.14*rad*rad*height;}
```

```
}
```

```
Cylinder::Cylinder(double x, double y, double r, double h):Circle(x,y,r){  
    height=h;
```

```
}
```

MULTIPLE DIRECT INHERITANCE

```
class Point{  
    double x;  
    double y;  
public:  
    Point(double x, double y){this->x=x; this->y=y;}  
    void get_xy(double &x, double &y){x=this->x, y=this->y;}  
}
```

MULTIPLE DIRECT INHERITANCE

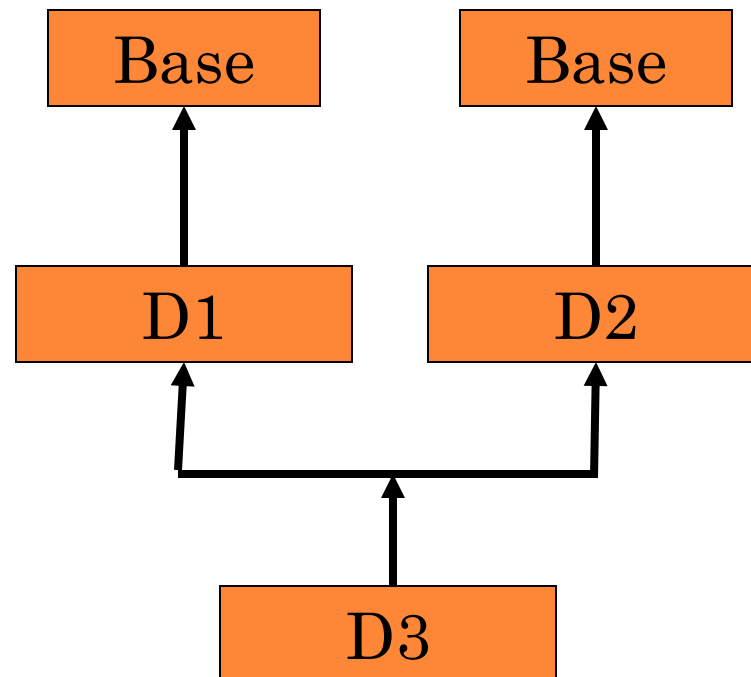
```
class Circle{  
protected:  
    double rad;  
public:  
    Circle(double r) {rad=r;}  
    double area(){return 3.14*rad*rad;}  
}
```

MULTIPLE DIRECT INHERITANCE

```
class Cylinder: public Point, public Circle{
    double height;
public:
    Cylinder(double x, double y, double r, double h);
    double volume(){ return 3.14*rad*rad*height;}
}
Cylinder::Cylinder(double x, double y, double r, double h):Point(x,y),Circle(r)
{
    height=h;
}
```

MULTIPLE INHERITANCE: VIRTUAL BASE CLASS

- Virtual Base Class prevents a derived class to inherit more than one copy of the base class
- This may happen when a derived class directly inherits two base classes and these base classes are also derived from another common base class



MULTIPLE INHERITANCE: VIRTUAL BASE CLASS

```
class Base {  
public:  
    int i;  
};  
class D1 : public Base {  
public:  
    int j;  
};  
class D2 : public Base {  
public:  
    int k;  
};
```

```
class D3 : public D1, public D2 {  
    // contains two copies of 'i'  
};  
  
void main() {  
    D3 obj;  
    obj.i = 10; // ambiguous, compiler error  
    obj.j = 20; // no problem  
    obj.k = 30; // no problem  
    obj.D1::i = 100; // no problem  
    obj.D2::i = 200; // no problem  
}
```

MULTIPLE INHERITANCE: VIRTUAL BASE CLASS

```
class Base {  
public:  
    int i;  
};  
class D1 : virtual public Base {  
public:  
    int j;  
}; // activity of D1 not affected  
class D2 : virtual public Base {  
public:  
    int k;  
}; // activity of D2 not affected
```

```
class D3 : public D1, public D2 {  
    // contains only one copy of 'i'  
}; // no change in this class definition  
  
void main() {  
    D3 obj;  
    obj.i = 10; // no problem  
    obj.j = 20; // no problem  
    obj.k = 30; // no problem  
    obj.D1::i = 100; // no problem, overwrites '10'  
    obj.D2::i = 200; // no problem, overwrites '100'  
}
```

FUNCTION OVERRIDING

- Giving new implementation of base class method into derived class is called function overriding
- Signature of base class method and derived class must be same
Signature involves:
 - Number of arguments
 - Type of arguments
 - Sequence of arguments

FUNCTION OVERRIDING

```
#include <iostream>
using namespace std;
class Point{
protected:
    double x;
    double y;
public:
    Point() {x=0.0; y=0.0;}
    Point(double x, double y);
    double area(){return 0;}
};

Point::Point(double x, double y){
    this->x=x; this->y=y;
}
```

FUNCTION OVERRIDING

```
class Circle: public Point{
protected:
    double rad;
public:
    Circle(){rad=0.0;}
    Circle(double x double y double r);
    double area();
};

Circle::Circle(double x, double y, double r) : Point(x,y) {
    rad=r;
}
double Circle::area(){
    return 3.14*rad*rad;
}
```

FUNCTION OVERRIDING

```
class Cylinder:public Circle{
    double height;
public:
    Cylinder(){height=0.0;}
    Cylinder(double x, double y, double r, double h);
    double area();
};
Cylinder::Cylinder(double x, double y, double r, double h): Circle(x,y,r){
    height=h;
}

double Cylinder::area(){
    return 3.14*rad*rad*height;
}
```

FUNCTION OVERRIDING

```
int main(){
```

```
    Point p(1.0, 1.0);
```

```
    Circle c(1.0, 1.0, 3.0);
```

```
    Cylinder cl(1.0, 1.0, 3.0, 2.0);
```

```
    cout<<"The area of the point is: "<<p.area()<<endl;
```

```
    cout<<"The area of the circle is: "<<c.area()<<endl;
```

```
    cout<<"The area of the cylinder is: "<<cl.area()<<endl;
```

```
    return 0;
```

```
}
```

POLYMORPHISM IN C++

- Compile time polymorphism
 - Uses static or early binding
 - Example: function and operator overloading
- Run time polymorphism
 - Uses dynamic or late binding
 - Example: virtual functions

EARLY BINDING VS. LATE BINDING

- Early binding
 - Normal functions, overloaded functions, nonvirtual member and friend functions
 - Resolved at compile time
 - Very efficient
 - But lacks flexibility
- Late binding
 - Virtual functions accessed via a base class pointer
 - Resolved at run-time
 - Quite flexible during run-time
 - But has run-time overhead; slows down program execution

POINTERS TO DERIVED CLASSES

- C++ allows base class pointers to point to derived class objects
- Let we have –
 - **class base { ... };**
 - **class derived : public base { ... };**
- Then we can write –
 - **base *p1; derived d_obj; p1 = &d_obj;**
 - **base *p2 = new derived;**

POINTERS TO DERIVED CLASSES

- Using a base class pointer (pointing to a derived class object) we can access only those members of the derived object that were inherited from the base
- This is because the base pointer has knowledge only of the base class
- It knows nothing about the members added by the derived class

POINTERS TO DERIVED CLASSES

```
class base {  
public:  
    void show() {  
        cout << "base\n";  
    }  
};  
class derived : public base {  
public:  
    void show() {  
        cout << "derived\n";  
    }  
};
```

```
void main() {  
    base b1;  
    b1.show(); // base  
    derived d1;  
    d1.show(); // derived  
    base *pb = &b1;  
    pb->show(); // base  
    pb = &d1;  
    pb->show(); // base  
}
```

- All the function calls here are statically/early bound

POINTERS TO DERIVED CLASSES

- While it is permissible for a base class pointer to point to a derived object, the reverse is not true

```
base b1;  
derived *pd = &b1; // compiler error
```

- We can perform a downcast with the help of type-casting, but should use it with caution
- Pointer arithmetic is relative to the data type the pointer is declared as pointing to
- If we point a base pointer to a derived object and then increment the pointer, it will not be pointing to the next derived object
- It will be pointing to (what it thinks is) the next base object !!!

VIRTUAL FUNCTION

- A virtual function is a member function that is declared within a base class and redefined (called ***overriding***) by a derived class
- It implements the “one interface, multiple methods” philosophy that underlies polymorphism
- The keyword **virtual** is used to designate a member function as virtual
- Supports run-time polymorphism with the help of base class pointers

VIRTUAL FUNCTION

- While redefining a virtual function in a derived class, the function signature must match the original function present in the base class
 - So, we call it overriding, not overloading
- When a virtual function is redefined by a derived class, the keyword virtual is not needed (but can be specified if the programmer wants)
- The “virtual”-ity of the member function continues along the inheritance chain
- A class that contains a virtual function is referred to as a polymorphic class

VIRTUAL FUNCTION

```
class base {  
public:  
    virtual void show() {  
        cout << "base\n";  
    }  
};  
class derived : public base {  
public:  
    void show() {  
        cout << "derived\n";  
    }  
};
```

```
void main() {  
    base b1;  
    b1.show(); // base - (s.b.)  
    derived d1;  
    d1.show(); // derived - (s.b.)  
    base *pb = &b1;  
    pb->show(); // base - (d.b.)  
    pb = &d1;  
    pb->show(); // derived (d.b.)  
}
```

- Here,
 - s.b. = static binding
 - d.b. = dynamic binding

VIRTUAL FUNCTION

```
class base {  
public:  
    virtual void show() {  
        cout << "base\n";  
    }  
};  
class d1 : public base {  
public:  
    void show() {  
        cout << "derived-1\n";  
    }  
};
```

```
class d2 : public base {  
public:  
    void show() {  
        cout << "derived-2\n";  
    }  
};  
void main() {  
    base *pb; d1 od1; d2 od2;  
    int n;  
    cin >> n;  
    if (n % 2) pb = &od1;  
    else pb = &od2;  
    pb->show(); // guess what ??  
}
```

VIRTUAL DESTRUCTORS

- Constructors cannot be virtual, but destructors can be virtual
- It ensures that the derived class destructor is called when a base class pointer is used while deleting a dynamically created derived class object

VIRTUAL DESTRUCTORS

```
class base {  
public:  
    ~base() {  
        cout << "destructing base\n";  
    }  
};  
class derived : public base {  
public:  
    ~derived() {  
        cout << "destructing derived\n";  
    }  
};
```

```
void main() {  
    base *p = new derived;  
    delete p;  
}
```

- Output:
 - destructing base

VIRTUAL DESTRUCTORS

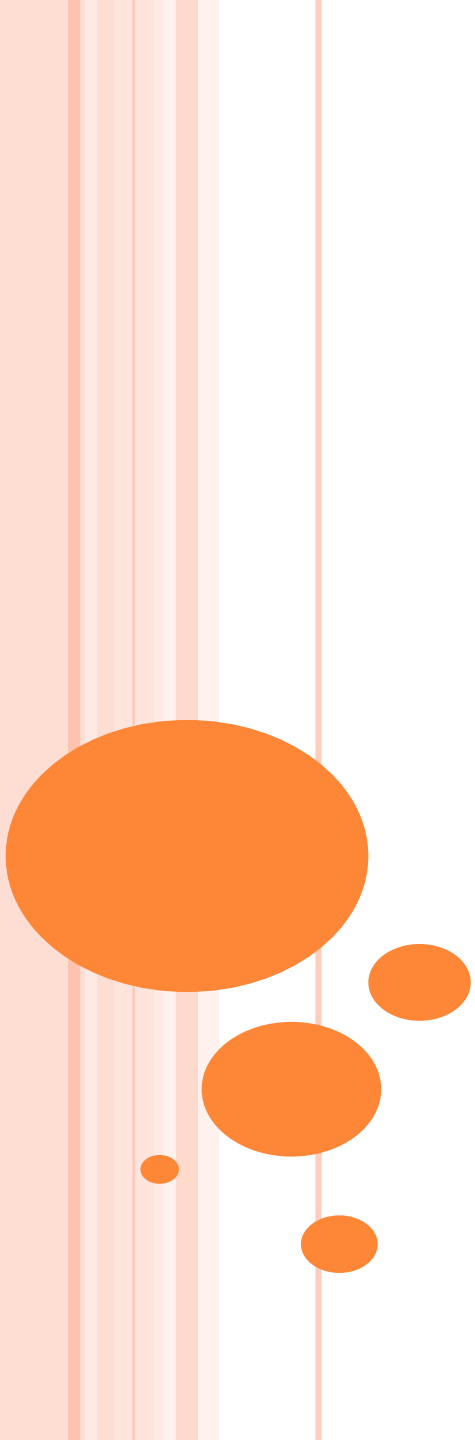
```
class base {  
public:  
    virtual ~base() {  
        cout << "destructing base\n";  
    }  
};  
class derived : public base {  
public:  
    ~derived() {  
        cout << "destructing derived\n";  
    }  
};
```

```
void main() {  
    base *p = new derived;  
    delete p;  
}
```

- Output:
 - destructing derived
 - destructing base

PURE VIRTUAL FUNCTIONS

- If we want to omit the body of a virtual function in a base class, we can use pure virtual functions
 - **virtual ret-type func-name(param-list) = 0;**
- It makes a class an *abstract class*
 - We cannot create any objects of such classes
- It forces derived classes to override it
 - Otherwise they become abstract too
- We can still create a pointer to an abstract class
 - Because it is at the heart of run-time polymorphism



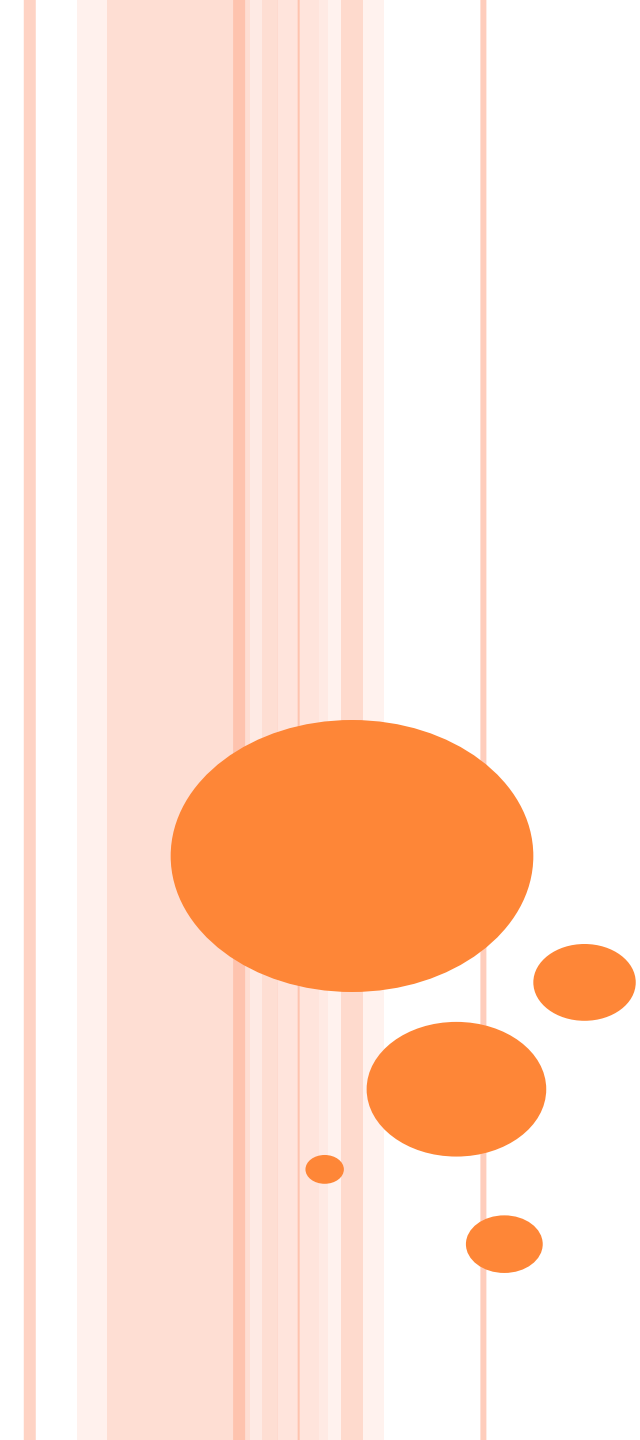
Acknowledgement

<http://faizulbari.buet.ac.bd/Courses.html>

<http://mhkabir.buet.ac.bd/cse201/index.html>

THE END

Topic Covered: Chapter 7 + Chapter 10



CSE 107: OBJECT ORIENTED PROGRAMMING LANGUAGE

Dr. Tanzima Hashem
Professor
CSE, BUET

C++ BASIC I/O

○ **Stream**

- a logical device that either produces or consumes information
- linked to a physical device

○ **ios:**

- contains many member functions and variables that control or monitor the fundamental operation of a stream
- **istream** and **ostream** inherit from **ios**
- **iostream** inherits from **istream** and **ostream**

C++ BASIC I/O

- << (left-shift operator)
 - Overloaded as *stream insertion (output) operator*
 - Associated with cout, a predefined stream linked to the console output (monitor)
- >> (right-shift operator)
 - Overloaded as *stream extraction (input) operator*
 - Associated with cin, a predefined stream linked to the console input (keyboard)
- In order to use either << or >> operator for console I/O programs must include iostream

CUSTOM INSERTER<<

```
#include <iostream>
using namespace std;
class myclass{
    int x, y;
public:
    myclass(int x, int y){this->x=x; this->y=y;}
    friend ostream &operator<<(ostream &out, myclass ob);
};
ostream &operator<<(ostream &out, myclass ob){
    out<<"x: "<<ob.x<<" , y: "<<ob.y<<endl;
    return out;
}
int main(){
    myclass ob(120, 130);
    cout<<ob;
    return 0;
}
```

CUSTOM EXTRACTOR >>

```
#include <iostream>
using namespace std;
class myclass {
    int x, y;
public:
    myclass (int x, int y){this->x=x; this->y=y;}
    friend ostream &operator<<(ostream &out, myclass ob);
    friend istream &operator>>(istream &in, myclass &ob);
};

ostream &operator<<(ostream &out, myclass ob){
    out<<"x: "<<ob.x<<" , y: "<<ob.y<<endl;
    return out;
}
```


CUSTOM EXTRACTOR >>

```
istream &operator>>(istream &in, myclass &ob){  
    cout<<"Enter x: ";  
    in>>ob.x;  
    cout<<"Enter y: ";  
    in>>ob.y;  
    return in;  
}  
  
int main(){  
    myclass ob(120, 130);  
    cout<<ob;  
    cin>>ob;  
    cout<<ob;  
    return 0;  
}
```

C++ FILE I/O

- A computer file is stored on a secondary storage device (e.g., disk)
 - permanent
 - can be used to
 - provide input data to a program
 - or receive output data from a program
 - or both;
 - should reside in project directory for easy access;
 - must be opened before it is used

C++ FILE I/O

- C++ provides three classes to perform output and input of characters to/from files:
 - **ofstream:** Stream class to write on files
 - **ifstream:** Stream class to read from files
 - **fstream:** Stream class to both read and write from/to files.
- These classes are derived directly or indirectly from the classes `istream` and `ostream`
- Done with the same operations (insertion, extraction) as keyboard input and monitor output
- Simply open input or output object with connection to a file and use it where you would use `cin` or `cout`

C++ FILE I/O

- To work with file you need to
 - include `<fstream>`
 - create input object of type *ifstream*
 - or output object of type *ofstream*

C++ FILE I/O

- A file needs to be opened by ifstream before read or ofstream to write or fstream to read and write
 - `void ifstream::open(const char *filename, openmode mode=ios::in)`
 - `void ofstream::open(const char *filename, openmode mode=ios::out | ios::trunc)`
 - `void fstream::open(const char *filename, openmode mode=ios::in | ios::out)`

C++ FILE I/O

- The value of mode determines how the file is opened
- Type **openmode** is an enumeration defined by **ios** that contains the following values:
 - `ios::app`
 - `ios::ate`
 - `ios::binary`
 - `ios::in`
 - `ios::out`
 - `ios::trunc`
- You can combine two or more of these value together by ORing them
- If `open()` fails the stream will evaluate to false

C++ FILE I/O: WRITING/READING

```
#include <iostream>
#include <fstream>
using namespace std;
int main () {
    ofstream myfile;
    myfile.open ("example.txt");
    myfile << "Writing this to a file.\n";
    myfile.close();
    return 0;
}
```

```
int main () {
    char ch;
    ifstream myfile ("example.txt");
    if (myfile.is_open()) {
        while ( !myfile.eof() ) {
            myfile>>ch;
            cout << ch;
        }
        cout<<endl;
        myfile.close();
    }
    else
        cout << "Unable to open file"

    return 0;
}
```

C++ FILE I/O (UNFORMATTED (BINARY))

○ **istream &get(char *ch*)**

- Member function of **fstream** and **ifstream**
- Associated stream must be opened with `ios::binary` openmode option
- Reads a single character from the stream and puts the value in *ch*
- Returns the reference to the stream
- If the end-of-file is reached returned stream will be evaluated false

C++ FILE I/O (UNFORMATTED (BINARY))

○ **ostream &put(char *ch*)**

- Member function of **fstream** and **ofstream**
- Associated stream must be opened with `ios::binary` openmode option
- Writes a single character from *ch* to the stream
- Returns the reference to the stream

C++ FILE I/O: WRITING/READING (UNFORMATTED (BINARY))

```
#include <iostream>
#include <fstream>
#include <cstdlib>
using namespace std;
int main(){
    char str[100]="I love Bangladesh";
    ofstream out("myfile.txt", ios::out | ios::binary);
    if(!out.is_open()){
        cout<<"Cannot open file"<<endl;
        exit(1);
    }
    for(int i=0; str[i]; i++)
        out.put(str[i]);
    out.close();
    return 0;
}
```

```
#include <iostream>
#include <fstream>
#include <cstdlib>
using namespace std;
int main(){
    char ch;
    ifstream in("myfile.txt", ios::in | ios::binary);
    if(!in.is_open()){
        cout<<"Cannot open file"<<endl; exit(1);
    }
    while(!in.eof()){
        in.get(ch);
        cout<<ch;
    }
    cout<<endl;
    in.close();
    return 0;
}
```

C++ FILE I/O: WRITING/READING (UNFORMATTED (BINARY))

```
#include <iostream>
#include <fstream>
#include <cstdlib>
using namespace std;
int main(){
    char str[100]="I love Bangladesh", ch;
    fstream mystream("myfile.txt", ios::out | ios::in | ios::binary);
    if(!mystream.is_open()){
        cout<<"Cannot open file"<<endl; exit(1);
    }
    for(int i=0; str[i]; i++) mystream.put(str[i]);
    mystream.seekg(0, ios::beg);
    while(!mystream.eof()){
        mystream.get(ch); cout<<ch;
    }
    cout<<endl;
    mystream.close();
    return 0;
}
```

C++ FILE I/O (UNFORMATTED (BINARY))

- **istream &read(char **buf*, streamsize *num*)**
 - Member function of **fstream** or **ifstream**
 - Reads *num* number of bytes from the stream and puts them in *buf*
 - Might read less than *num* number of bytes if the end-of-file is reached ahead
 - How many bytes have been read can be determined by using **gcount()** member function
- **ostream &write(const char **buf*, streamsize *num*)**
 - Member function of **fstream** or **ofstream**
 - Writes *num* number of bytes from *buf* in the stream

C++ FILE I/O (UNFORMATTED (BINARY))

- Overloaded **get()** member function of **fstream** or **ifstream**
 - **istream &get(char **buf*, streamsize *num*)**
 - **istream &get(char **buf*, streamsize *num*, char *delim*)**
 - **int get()**

C++ FILE I/O (UNFORMATTED (BINARY))

- **istream &get(char **buf*, streamsize *num*)**
 - Reads *num-1* characters into *buf* until
 - *num-1* characters have been read
 - a newline is found
 - the end of the file has been encountered
 - Character sequence in the *buf* is null terminated
 - If newline character is encountered, it is not read and removed from the stream
- **istream &get(char **buf*, streamsize *num*, char *delim*)**
 - Reads *num-1* characters into *buf* until
 - *num-1* characters have been read
 - a *delim* character is found
 - the end of the file has been encountered
 - Character sequence in the *buf* is null terminated
 - If *delim* character is encountered, it is not read and removed from the stream

C++ FILE I/O (UNFORMATTED (BINARY))

- **int get()**
 - Returns the next character from the stream
 - Returns EOF if end-of-file is encountered
 - Similar to C's getc() function.

C++ FILE I/O (UNFORMATTED (BINARY))

- Overloaded **getline()** member function of **fstream** or **ifstream**
- **istream &getline(char **buf*, streamsize *num*)**
- **istream &getline(char **buf*, streamsize *num*, char *delim*)**
- Same as **get**, except extract and remove the newline or **delim** character from the input stream

C++ FILE I/O (UNFORMATTED (BINARY))

- **istream &putback(char ch)**

- Puts **ch** back to the input stream so the next extracted character will be **ch**

- **int peek()**

- Returns the next character in the input stream without removing it

- **ostream &flush()**

- Force the information to be physically written to disk before the internal buffer is full

C++ FILE I/O (UNFORMATTED (BINARY))

RANDOM ACCESS

- **istream &seekg(off_type *offset*, seekdir *origin*)**
 - A member function of **istream** and **fstream**
 - Moves the current get pointer *offset* number of bytes from the specified *origin*
- **ostream &seekp(off_type *offset*, seekdir *origin*)**
 - A member function of **ostream** and **fstream**
 - Moves the current put pointer *offset* number of bytes from the specified *origin*
- **seekdir** is an enumeration defined in **ios** with the following values:
 - **ios::beg**
 - **ios::cur**
 - **ios::end**

C++ FILE I/O (UNFORMATTED (BINARY))

RANDOM ACCESS

```
#include <fstream>
#include <iostream>

using namespace std;

int main (int argc, char** argv)
{
    fstream myFile("test.txt", ios::in | ios::out | ios::trunc);

    myFile << "Hello World";

    myFile.seekg(6, ios::beg);
```

```
char A[6];
myFile.read(A, 5);
A[5] = '\0';

cout << A << endl;

myFile.close();
}
```

Output:
World

C++ FILE I/O (UNFORMATTED (BINARY))

RANDOM ACCESS

- **pos_type tellg()**
 - Returns current position of get pointer
- **pos_type tellp()**
 - Returns current position of put pointer
- **istream &seekg(pos_type position)**
- **ostream &seekp(pos_type position)**
 - Overloaded versions of seekg and seekp

C++ FILE I/O (UNFORMATTED (BINARY))

RANDOM ACCESS

```
#include <fstream>
using namespace std;

int main()
{
    long position;
    fstream file;

    file.open("myfile.txt");

    file.write("this is an apple", 16);
    position = file.tellp();

    file.seekp(position - 7);
    file.write(" sam", 4);
    file.close();
}
```

Output:
this is a sample

C++ FILE I/O

CHECKING THE STATUS

- C++ I/O System maintains status information about the outcome of each I/O operation
- The current status of an I/O stream is described in an object of type **iosstate**, which is an enumeration defined in **ios** with the following values:
 - **ios::goodbit**
 - **ios::eofbit**
 - **ios::failbit**
 - **ios::badbit**
- To read the I/O status you can use **rdstate()** function
 - **iosstate rdstate()**
 - It is a member of **ios** and inherited by all the streams

C++ FILE I/O

CHECKING THE STATUS

```
#include <iostream>
#include <fstream>
#include <cstdlib>
using namespace std;
void checkstatus(ifstream &in);
int main(){
    char ch;
    ifstream in("myfile.txt");
    if(!in.is_open()){
        cout<<"Cannot open file"<<endl;
        exit(1);
    }
    while(!in.eof()){
        ch=in.get();
        checkstatus(ifstream &in);
        cout<<ch;
    }
```

```
        cout<<endl;
        in.close();
        return 0;
    }

void checkstatus(ifstream &in){
    ios::iostate s;
    s=in.rdstate();
    if(s&ios::eofbit)
        cout<<"EOF encountered"<<endl;
    else if(s&ios::failbit)
        cout<<"Non-Fatal I/O error encountered"<<endl;
    else if(s&ios::badbit)
        cout<<"Fatal I/O error encountered"<<endl;
    }
```

C++ FILE I/O

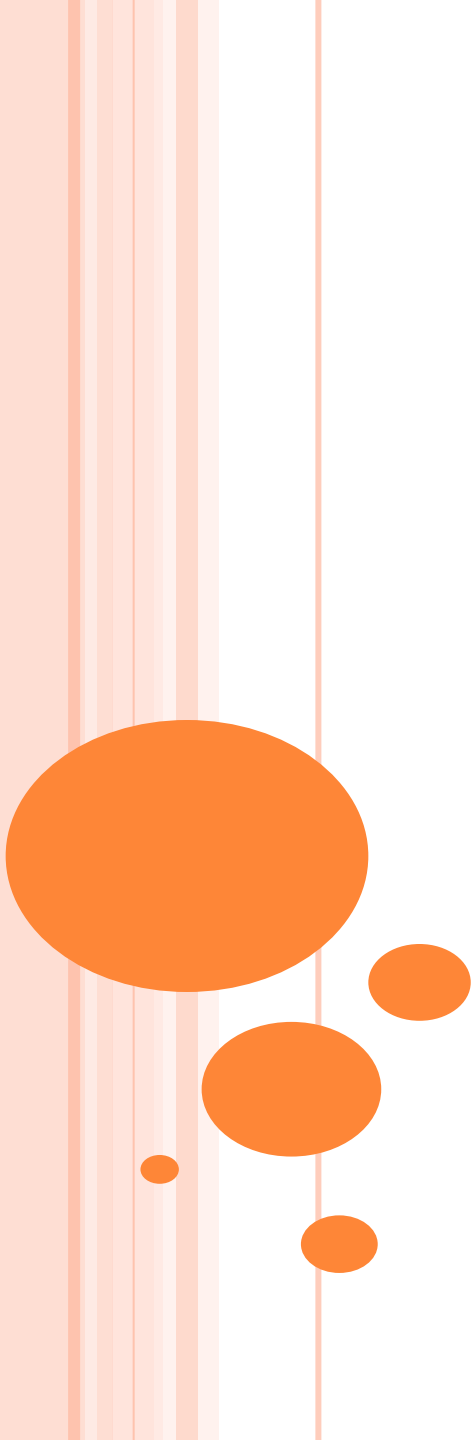
CHECKING THE STATUS

- The status can be determined by using following **ios** member functions those have also been inherited by all the streams
 - **bool eof()**
 - **bool good()**
 - **bool fail()**
 - **bool bad()**

C++ FILE I/O

CHECKING THE STATUS

```
while(!in.eof()){  
    ch=in.get();  
    if(in.fail() || in.bad()){  
        cout<<"I/O Error ... terminating"<<endl;  
        return 1;  
    }  
    cout<<ch;  
}
```



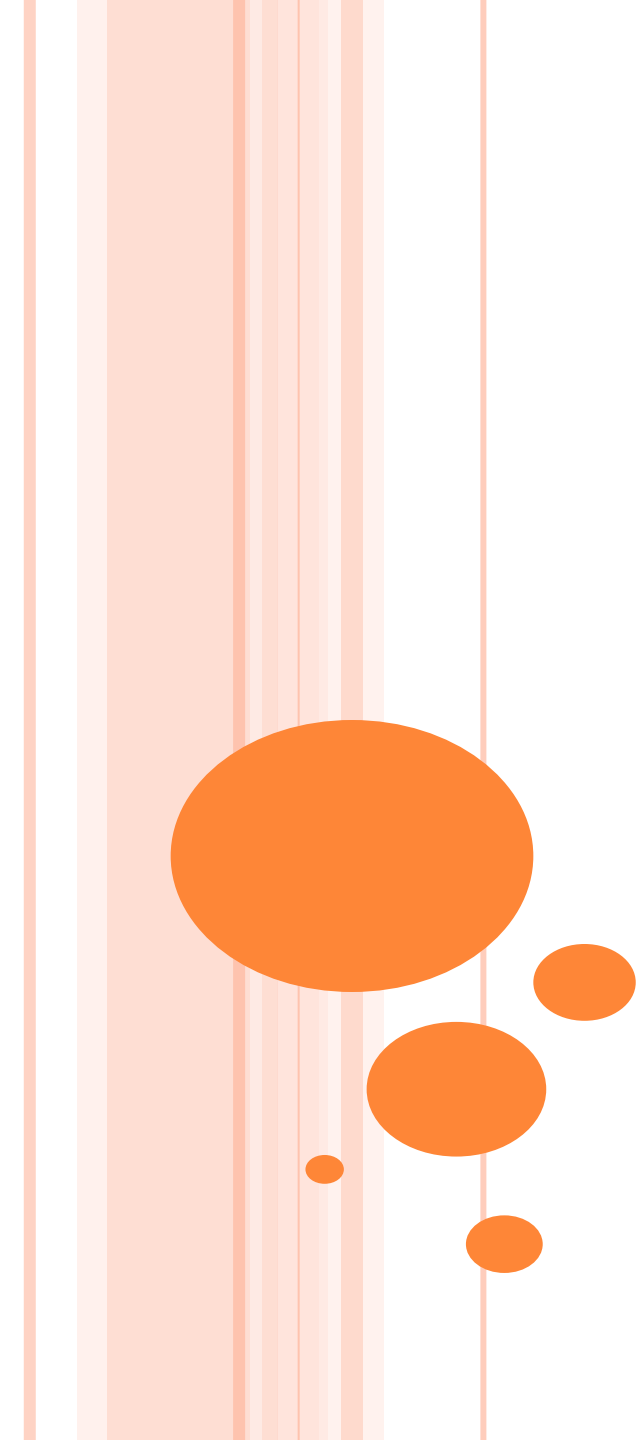
Acknowledgement

<http://faizulbari.buet.ac.bd/Courses.html>

<http://mhkabir.buet.ac.bd/cse201/index.html>

THE END

Topic Covered: Sections 8.5, 8.6, 9.2, 9.3, 9.4, 9.5, 9.6



CSE 107: OBJECT ORIENTED PROGRAMMING LANGUAGE

Dr. Tanzima Hashem
Professor
CSE, BUET

STATIC CLASS MEMBERS

- Static member variable
 - Only one copy of that variable to exist-no matter how many objects of that class are created
 - All objects simply share that one variable
 - Also, the same static variable will be shared by any class derived from the class that contains the static member
 - Exists before any of its object is created
 - A global variable that simply has its scope restricted to the class in which it is declared

STATIC CLASS MEMBERS

- A static member variable is defined with the keyword static before the data-type of the variable
- To ensure that the storage for a static member is allocated it has to be defined second time outside the class using the class-name and scope operator
- A static member variable is initialized to zero by default
- A static member variable can be accessed
 - through the objects of the class or
 - directly through the class-name and scope resolution operator

STATIC CLASS MEMBERS

```
#include <iostream>
using namespace std;
class myclass{
    static int i;
public:
    void seti(int x){i=x;}
    int geti(){return i;}
};

int myclass::i;
```

```
int main(){
    myclass ob1, ob2;
    ob1.seti(200);
    cout<<ob1.geti()<<endl;
    //Prints 200
    cout<<ob2.geti()<<endl;
    //Prints 200
    return 0;
}
```

STATIC CLASS MEMBERS

```
#include <iostream>
using namespace std;
class myclass{
    static int i;
public:
    void seti(int x){i=x;}
    int geti(){return i;}
};

int myclass::i=500; // allowed
```

```
int main(){
    myclass ob1, ob2;
    //cout<<myclass::i<<endl; -error
    cout<<ob1.geti()<<endl;
    ob1.seti(200);
    cout<<ob1.geti()<<endl;
    //Prints 200
    cout<<ob2.geti()<<endl;
    //Prints 200
    return 0;
}
```

STATIC CLASS MEMBERS

```
#include <iostream>
using namespace std;
class myclass{

public:
    static int i;
    void seti(int x){i=x;}
    int geti(){return i;}
};

int myclass::i=100;
```

```
int main(){
    myclass ob1, ob2;
    cout<<myclass::i<<endl;
    //Prints 100
    ob1.seti(200);
    cout<<ob1.geti()<<endl;
    //Prints 200
    cout<<ob2.geti()<<endl;
    //Prints 200
    return 0;
}
```


STATIC CLASS MEMBERS

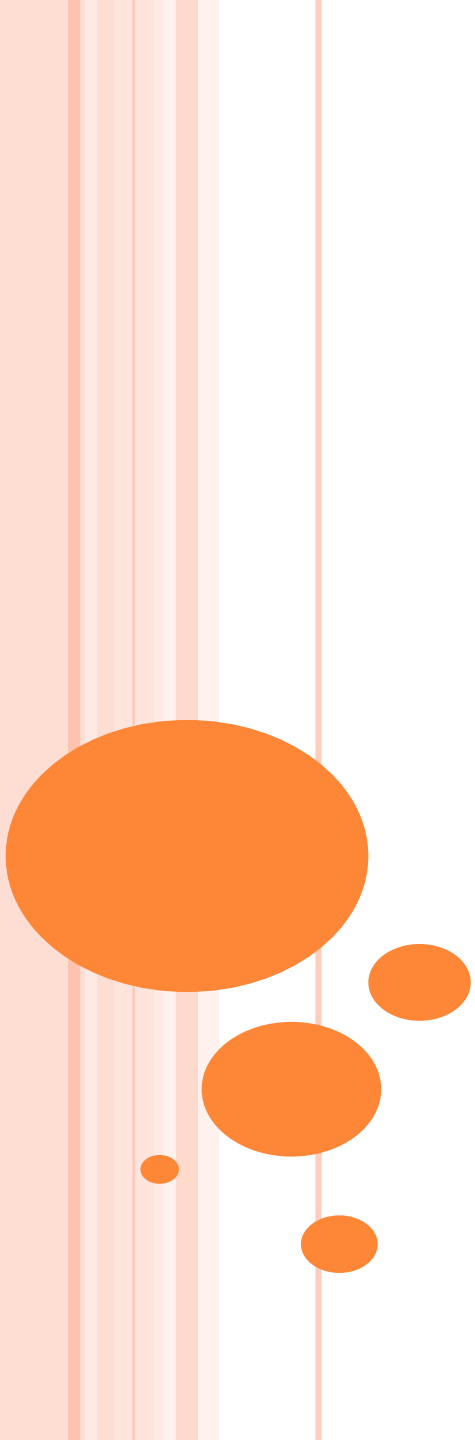
- Static member function
 - Access only other static members of its class
 - Access global (non-static) variables and functions
- A static member function does not have a this pointer since only one copy of the function is shared by all the objects of the class
- A static member function can be accessed
 - through the objects or
 - through the class directly without creating any object
- Virtual static member function is not allowed
- A static member function cannot be const and volatile

STATIC CLASS MEMBERS

```
#include <iostream>
using namespace std;
class myclass{
    static int i;
    public:
    static void seti(int x){i=x;}
    int geti(){return i;}
};

int myclass::i;
```

```
int main(){
    myclass ob1, ob2;
    ob1.seti(200);
    cout<<ob1.geti()<<endl;
    //Prints 200
    cout<<ob2.geti()<<endl;
    //Prints 200
    myclass::seti(300);
    cout<<ob1.geti()<<endl;
    //Prints 300
    cout<<ob2.geti()<<endl;
    //Prints 300
    return 0;
}
```



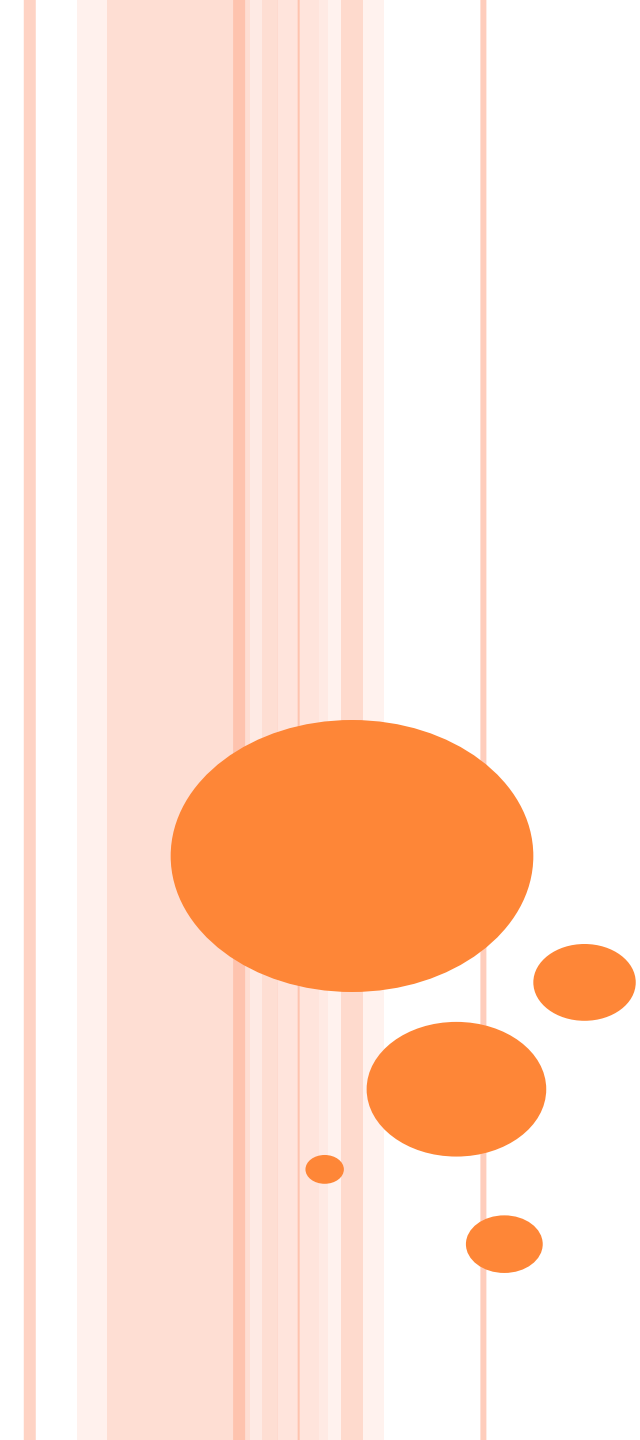
Acknowledgement

<http://faizulbari.buet.ac.bd/Courses.html>

<http://mhkabir.buet.ac.bd/cse201/index.html>

THE END

Topic Covered: Section 13.3



CSE 107: OBJECT ORIENTED PROGRAMMING LANGUAGE

Dr. Tanzima Hashem
Professor
CSE, BUET

STRING

- A string is a sequence of character
- We have used null terminated `<char>` arrays (C-strings or C-style strings) to store and manipulate strings
 - We include `<cstring>` to use C-style string functions, such as `strlen`, `strcpy` etc
- C++ provides a class called **string**
 - We include `<string>` in our program to create objects of string class

AVAILABLE OPERATIONS

- Creating string objects
- Reading string objects from keyboard
- Displaying string objects to the screen
- Finding a substring from a string
- Modifying string objects
- Adding string objects
- Accessing characters in a string
- Obtaining the size of string
- And many more

COMMONLY USED STRING CONSTRUCTORS

- `String();`
 // For creating an empty string.
- `String(const char *str);`
 // For creating a string object from a null-terminated string
- `String(const string &str);`
 // For creating a string object from other string object

OPERATOR OVERLOADED IN STRING CLASS

Operator	Meaning
=	Assignment
+	Concatenation
+=	Concatenation assignment
==	Equality
!=	Inequality
<	Less than
<=	Less than equal

OPERATOR OVERLOADED IN STRING CLASS

Operator	Meaning
>	Greater than
>=	Greater than equal
[]	Subscripting
<<	Output
>>	Input

CREATING STRING OBJECTS AND OPERATIONS

- `string s1, s3;` `// Using constructor with no arguments`
- `string s2("xyz");` `// Using one-argument constructor`
- `string s4(s1)` `// Create a string object with another string object`
- `s1 = s2;` `// Assigning string objects`
- `cin >> s1;` `// Reading from keyboard (one word)`
- `cout << s2;` `// Display the content of s2`
- `getline(cin, s1)` `// Reading from keyboard a line of text`
- `s3 = "abc" + s2;` `// Concatenating strings`
- `s3 += s1;` `// s3 = s3 + s1;`
- `s3 += "abc";` `// s3 = s3 + "abc";`

MANIPULATING STRING OBJECTS

- assign

- `string &assign(const string &strob, size_type start, size_type num)`

- `string str;`

- `string base="The quick brown fox jumps over a lazy dog.";`

- `str.assign(base,10,9);`

- `"brown fox"`

- `string &assign(const char *str, size_type num)`

- `string str;`

- `str.assign("pangrams are cool",7);`

- `"pangram"`

MANIPULATING STRING OBJECTS

- `append()`
 - `string &append(const string &strob, size_type start, size_type num)`
 - `string str;`
 - `str3="print 10 and then 5 more"`
 - `str.append(str3,6,3);`
 - `"10 "`
 - `string &append(const char *str, size_type num)`
 - `string str;`
 - `str.append("dots are cool",5);`
 - `"dots "`

MANIPULATING STRING OBJECTS

- insert()
 - `string &insert(size_type, start, const string &t b)`
 - `string str="to be question";`
 - `string str2="the ";`
 - `str.insert(6,str2);`
 - “to be the question”
 - `string &insert(size_type, start, const string &strob, size_type inStart, size_type num)`
 - `string str="to be question";`
 - `str3="or not to be";`
 - `str.insert(6,str3,3,4);`
 - “to be not question”

MANIPULATING STRING OBJECTS

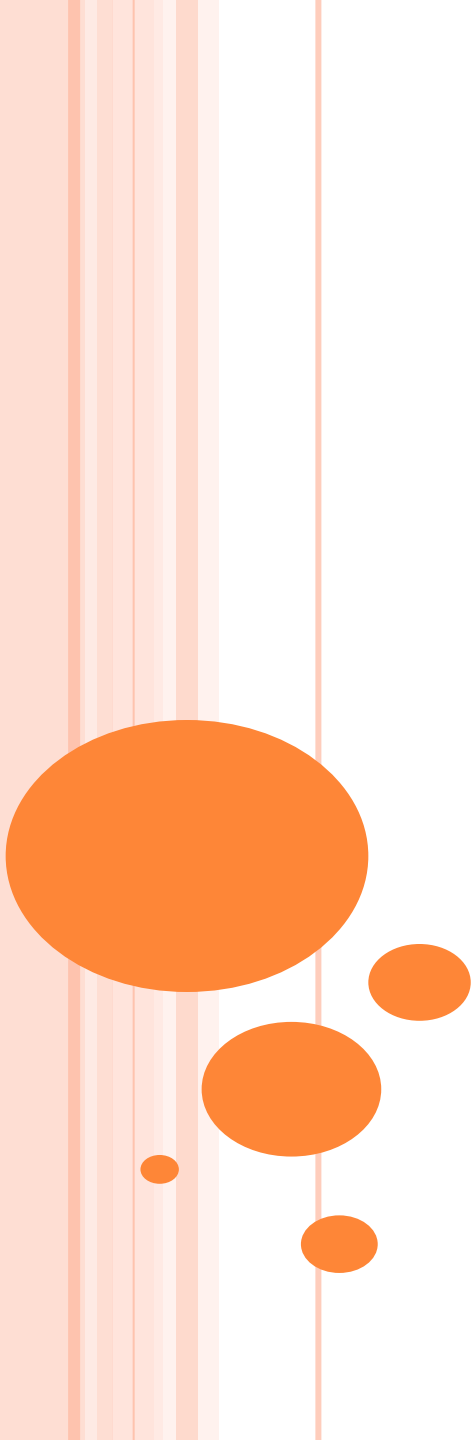
- `replace()`
 - `string &replace(size_type start, size_type num, const string &strob)`
 - `string str="this is a test string.";`
 - `string str2="n example";`
 - `str.replace(9,5,str2);`
 - `"this is an example string."`
 - `string &replace(size_type start, size_type num, const string &strob, size_type replaceStrat, size_type replaceNum)`
 - `string str="this is an example string.";`
 - `string str3="sample phrase";`
 - `str.replace(19,6,str3,7,6);`
 - `"this is an example phrase."`

MANIPULATING STRING OBJECTS

- `erase()`
 - `string &erase(size_type start=0, size_type num=npos) //npos= -1`
 - `string str ("This is an example sentence.");`
 - `str.erase (10,8);`
 - `"This is an sentence."`

RELATIONAL OPERATIONS

- `string s1("ABC"); string s2("XYZ");`
- `int x = s1.compare(s2);`
 - `x == 0` if `s1 == s2`
 - `x > 0` if `s1 > s2`
 - `x < 0` if `s1 < s2`



Acknowledgement

<http://faizulbari.buet.ac.bd/Courses.html>

<http://mhkabir.buet.ac.bd/cse201/index.html>

THE END

Topic Covered: Sections 14.7 (Except Example 3)